

Privately Constrained PRFs from DCR: Puncturing and Bounded Waring Rank

Amik Raj Behera¹, Pierre Meyer², Claudio Orlandi², Lawrence Roy², and Peter Scholl²

¹ University of Copenhagen, Copenhagen, Denmark ambe@di.ku.dk

² Aarhus University, Aarhus, Denmark

[pierre.meyer,orlandi,peter.scholl}@cs.au.dk](mailto:{pierre.meyer,orlandi,peter.scholl}@cs.au.dk), ldr709@gmail.com

Abstract. A *privately constrained pseudorandom function (pCPRF)* is a PRF with the additional property that one can derive a constrained key that allows evaluating the PRF only on inputs satisfying a constraint predicate C , without revealing C itself or leaking information about the PRF’s output on inputs that do not satisfy the constraint.

Existing privately constrained PRFs face significant limitations: either (1) they rely on assumptions known to imply fully-homomorphic encryption or indistinguishability obfuscation, (2) they support only highly restricted classes of constraints—notably, no known group-based pCPRF even supports the simple class of *puncturing* constraints (where the constrained key permits evaluation on all but one point while hiding the punctured point), or (3) they are limited to polynomial-size input domains. A long-standing open question has been whether one can construct a privately constrained PRF from group-based assumptions for more expressive classes of constraints. In this work, we present a pCPRF based on the decisional composite residuosity (DCR) assumption that supports a highly expressive class of predicates, namely constraints whose degree and Waring rank are both polynomially bounded, which includes puncturing.

From a technical perspective, our work follows the general template of Couteau, Meyer, Pas-selègue, and Riahinia (Eurocrypt’23), who constructed a pCPRF from group-based homomor-phic secret-sharing but were limited to inner-product constraints in the constraint-hiding set-ting. Leveraging novel techniques for computing with *distributed discrete logarithms (DDLog)*, we enable the non-interactive authentication of powers of linear combinations of shares of some value. This, in turn, allows us to express constraints with polynomially bounded Waring rank.

Our construction is single-key, selectively secure, and supports an exponential-size domain.

Table of Contents

1	Introduction	3
2	Technical Overview	4
2.1	The CMPR Template.	5
2.2	Our Solution: Authenticating Powers of Shares.....	7
2.3	Waring Rank	9
3	Preliminaries	10
3.1	Damgård-Jurik-ElGamal Cryptosystem	10
3.2	Distributed Discrete Logarithm	12
3.3	Privately Constrained Pseudorandom Function.....	12
4	Share Conversion Procedures	13
4.1	Homomorphic Share Conversion: Multiplication	14
4.2	Homomorphic Share Conversion: Bounded Waring Rank	15
4.3	Homomorphic Share Conversion: RMS Programs with Leading Multiplication .	18
5	Privately Constrained PRF for Bounded Dual Waring Rank	20
6	Waring Rank	23
A	More Preliminaries	30
A.1	Damgård-Jurik-ElGamal encryption scheme	30
A.2	Damgård-Jurik-ElGamal Cryptosystem Proofs	31
A.2.1	Correctness of Damgård-Jurik-ElGamal	31
A.2.2	DCR over $(\mathbb{Z}/N^{\zeta+1}\mathbb{Z})^\times$	31
A.2.3	KDM Security	32
A.3	Distributed Discrete Logarithm	33
B	Proofs	34
B.1	Proof of core lemma 1	34
B.2	Proof of main theorem 1	37
B.2.1	Correctness lemma.....	37
B.2.2	Full simulation-based proof.	37

1 Introduction

Pseudorandom functions (PRFs) are a fundamental tool in both the theory and practice of cryptography. A constrained PRF (CPRF) allows a master secret PRF key to be used to derive a constrained key that permits evaluation of the PRF only on a restricted set of inputs. This restriction is determined by a predicate C , such that the constrained key allows evaluation at all points where $C(x) = 0$, while for any x where $C(x) = 1$, the PRF output remains pseudorandom.³

Some CPRFs support *private constraints* (or are constraint-hiding), meaning that the constrained key reveals no information about the specific constraint used, beyond the allowed class of constraints.

Originally, constrained PRFs were proposed for applications such as identity-based key exchange and broadcast encryption [BW13]. Later, they played a key role in the punctured programming technique [SW14], which enabled many applications of indistinguishability obfuscation. CPRFs have also been used to construct watermarkable PRFs, searchable encryption, private keyword search, and symmetric-key deniable encryption [BLW17].

One of the most fundamental constraint classes is *puncturing*, where $C(x) = 1$ if and only if $x = x^*$. In this case, a constrained key—called a punctured key—allows evaluation of the PRF at all points except for x^* . Puncturable PRFs can be built using the GGM construction [GGM84], where a PRF is constructed from a tree of length-doubling pseudorandom generators (PRGs). However, in GGM-style constructions [BW13, KPTZ13, BGI14], the punctured key reveals the punctured point x^* , meaning that the construction is not private.

Privately puncturable PRFs were first constructed by Boneh et al. [BLW17] using multilinear maps or indistinguishability obfuscation, achieving constrained PRFs for general polynomial-size circuits. Later constructions obtained privately constrained PRFs for circuits under the learning with errors (LWE) assumption [BTW17, CC17] and were further extended in various ways, such as enabling programmability [PS18] and adaptive security [HKKW19, DKN⁺20]. Alternative constructions support weaker classes of predicates or security properties while avoiding the full power of LWE. These include constructions based on DDH [AMN⁺18], DCR [CMPR23], and even solely one-way functions [DKN⁺20, Ser24].

Notably, however, none of these constructions achieve a *private* puncturable PRF without relying on either LWE or indistinguishability obfuscation. The only exception is the work of Boyle et al. [BGIK22], which is based solely on one-way functions but requires the PRF domain to be polynomially sized (as generating a constrained key requires iterating over the entire domain). Thus, this construction is more akin to a “privately programmable incremental PRG⁴.” This leaves the following challenging open problem:

Is it possible to construct a privately puncturable PRF with a superpolynomial domain size without relying on LWE or indistinguishability obfuscation?

Our Contributions. We construct the first privately puncturable PRF from the decisional composite residuosity (DCR) assumption that supports an exponentially large domain, resolving the question posed above. In fact, we construct a private CPRF for a broader class of functions—namely, functions with polynomially bounded Waring rank—that includes puncturing.

³ The convention of using $C(x) = 0$ (instead of $C(x) = 1$) for satisfying inputs simplifies the exposition of our construction.

⁴ We say a PRG is incremental (sometimes called *iterative* [BCG⁺19]) if each output bit can be computed in time polylogarithmic in the seed size. This is in contrast with a PRG which might require a “full evaluation” in order to recover a given bit of output.

A function has polynomially bounded Waring rank if it can be expressed as the sum of at most polynomially many powers of linear functions, and has polynomial degree. This class encompasses many useful functions, including puncturing, as well as all log-local functions, inner product membership [BCM⁺24], and more.

Our private CPRF construction follows the homomorphic secret-sharing-based template for building private CPRFs introduced by Couteau, Meyer, Passelègue, and Riahinia [CMPR23]. In essence, the CPRF key and constrained keys are defined as the values needed for non-interactively computing (subtractive) secret shares of $C(x) \cdot F_K(x)$, where C represents the constraint and F is a PRF. Notably, the CPRF outputs are the *shares* of this value, and the PRF F is not the PRF being constrained, but rather a tool for ensuring that when $C(x) = 1$, the shares remain pseudorandom. Specifically, when $C(x) = 0$, both parties hold the same shares (i.e., the constrained key produces the same output as the master key), whereas when $C(x) = 1$, the PRF F ensures that the two shares are pseudorandom.

The construction of [CMPR23] supports only constraints that can be expressed as inner products, as it relies on a special form of “offline-online” homomorphic secret sharing—a two-party, authenticated secret-sharing scheme where one party’s share x_0 of an input x can be generated independently of x , and the other share x_1 is later generated using x_0 and x . Due to this restriction, offline-online shared values support only linear homomorphism, limiting their CPRF construction to inner-product constraints.⁵ We overcome this limitation via a new technique based on distributed discrete logarithms (DDLog) and novel methods for homomorphically combining authenticated secret shares. This technique may be of independent interest and lead to further applications.

Comparison. There is a vast body of literature on constructing constrained PRFs under various cryptographic assumptions. In Table 1, we summarise prior and concurrent results that rely on group-based assumptions or one-way functions, as these are most relevant to our work. The most expressive prior constructions are those of [AMN⁺18], which is based on a variant of DDH in pairing-free groups, and [CMPR23], which relies on DCR or class group assumptions. Both of these constructions support arbitrary NC1 constraints; however, neither achieves constraint hiding.

Previous private CPRFs with superpolynomial domain sizes support constraints such as inner products, bit-fixing, or t -CNF for constant $t = O(1)$. Despite their apparent generality, none of these function classes are expressive enough to capture puncturing, which is inherently a high-degree constraint.

Concurrent Work. Ishai, Li, and Lin [ILL24] concurrently⁶ developed new CPRF constructions extending the CMPR template. Their work introduces techniques for succinct partial garbling, allowing them to construct CPRFs for any constraint expressible by a circuit. However, because their approach relies on partial garbling, their construction does not achieve privacy. Thus, our results and those of [ILL24] extend the CMPR template in different directions. An interesting open question is whether both approaches can be combined to achieve a private CPRF for general circuits.

2 Technical Overview

We write $\langle x \rangle$ for *subtractive shares* of x , and we denote by $\langle x \rangle_\sigma$ the share held by party P_σ . We call $\langle \text{sk} \cdot x \rangle$ an *authenticated share* of x . Similarly, we refer to $\text{Enc}_{\text{sk}}(\text{sk} \cdot x)$ as an *authenticated encryption* of x .

⁵ [CMPR23] also obtained a CPRF for NC1 circuits, but this construction is not private.

⁶ Concurrency has been established via private communication.

Table 1: Constructions of CPRFs using group-based assumptions or one-way functions. All selectively secure constructions can be made adaptively secure in the random oracle model.

	Private	Constraints	Adaptive	Domain Size	Assumption
[BW13], [KPTZ13], [BGI14]	✗	Prefix/puncturing	✗	exp	OWF
[BW13]	✗	Left-right	✓	exp	Bilinear DDH + ROM
[AMN ⁺ 18]	✗	NC1	✗	exp	L-DDHI
[CMPR23]	✗	NC1	✗	exp	DCR/class groups
[ILL24]	✗	P/Poly	✗	exp	DCR/class groups
[AMN ⁺ 18]	✓	Bit-fixing	✗	exp	DDH
[DKN ⁺ 20]	✓	t -CNF	✓	exp	OWF
[CMPR23]	✓	Inner product	✗	exp	DCR/class groups
[Ser24]	✓	Inner product	✓	exp	ROM
[Ser24]	✓	Inner product	✗	exp	DDH
[Ser24]	✓	Inner product	✗	poly	OWF
[BGIK22]	✓	Puncturing	✗	poly	OWF
Ours	✓	Bounded Waring Rank (incl. Puncturing)	✗	exp	DCR

2.1 The CMPR Template.

Couteau, Meyer, Passelègue, and Riahiinia [CMPR23] provided a template for building (privately) constrained PRFs using homomorphic secret sharing (HSS) *techniques*⁷. The key idea is that the task can be viewed as enabling two parties—the master key holder and the constrained key holder—to compute pseudorandom shares of the value $C(x) \cdot \text{PRF}_k(x)$ for any input x of the CPRF. If $C(x) = 0$ (meaning the input x is authorised), both shares are equal, whereas if $C(x) = 1$, they are offset by $\text{PRF}_k(x)$. Provided the constrained key does not reveal k , this ensures that unauthorised evaluations remain masked.

Homomorphic secret sharing supporting a class of functions $\mathcal{F}$ allows two parties to share a secret value x (with computational security), and then locally evaluate these shares with respect to any function $f \in \mathcal{F}$ and retrieve additive shares of $f(x)$. If the parties generate HSS shares of some PRF key k (since this is constraint-independent, these shares can be generated as part of the master key, and then the second party’s share can be made part of the constrained key when it is generated), they can evaluate the function $X \mapsto \text{PRF}_X(x)$ to get shares of $\text{PRF}_k(x)$. The starting point for the template is that group-based HSS schemes following the “BGI template” [BGI16] support this class of computation and have an additional (non black-box property): these HSS schemes have the parties hold additive shares of some secret key sk , but if these shares are replaced with shares of $\text{sk} \cdot Y$ (where Y is some scalar), then the HSS evaluation step can be tweaked so the parties compute additive shares of $Y \cdot \text{PRF}_k(x)$ (instead of $\text{PRF}_k(x)$). The question of building a *privately* constrained PRF is therefore reduced to the question of allowing the parties to generate shares of $\text{sk} \cdot C(x)$ for *any* one of exponentially many inputs x , while satisfying the two following requirements:

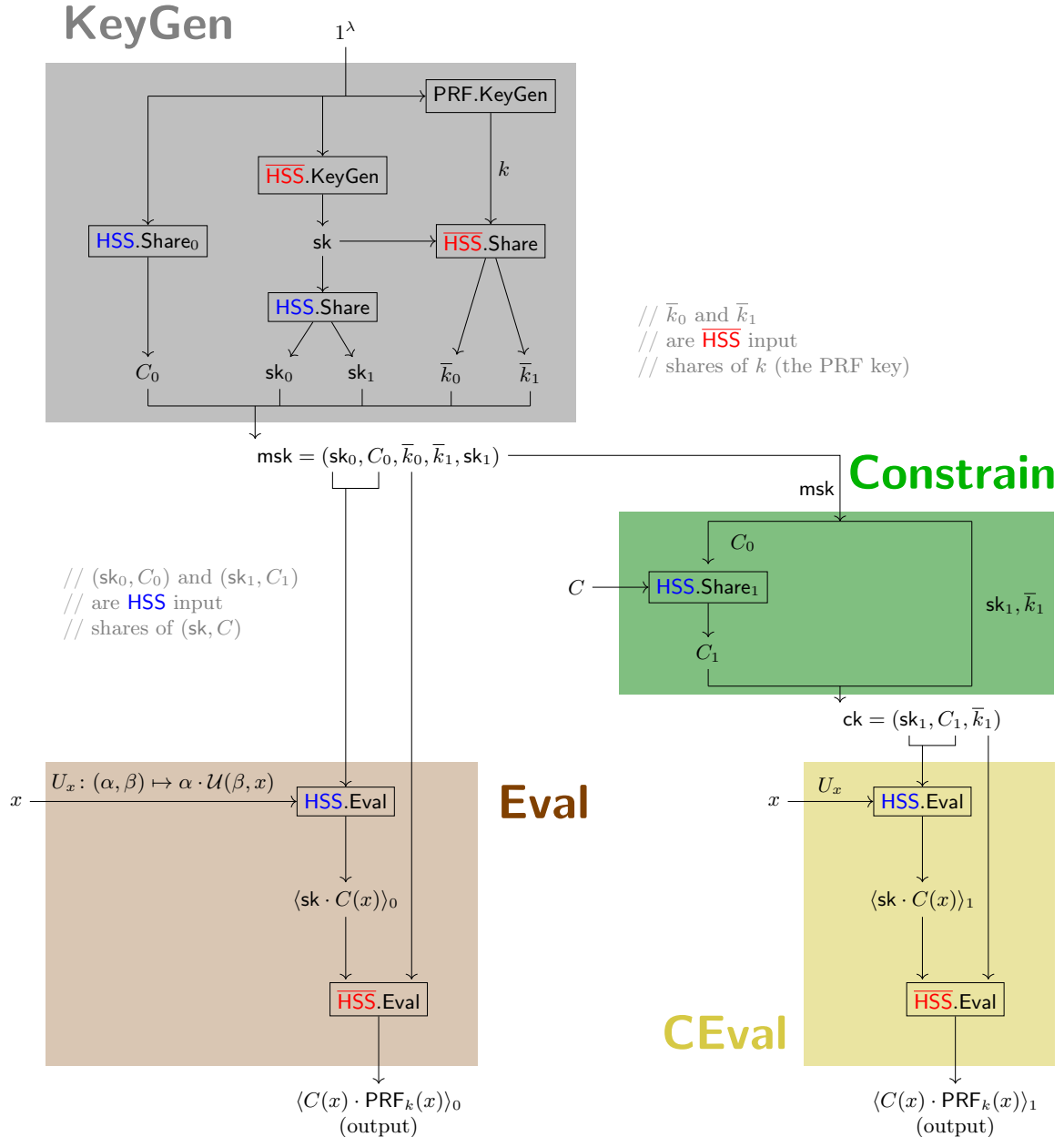
1. the first party’s material (the master key) should be generated without knowledge of C ;
2. the second party’s material (the constrained key) should computationally hide C .

Prior to this work, the only known way to do this was to use a linearly homomorphic, information-theoretic, additive secret sharing scheme [CMPR23], but this approach is limited

⁷ We emphasise that the template is highly non-black box in HSS, and uses additional ad-hoc properties for the construction to go through.

to inner-product constraints: if the constraint is defined as $C_Z: x \mapsto \text{IP}(Z, x)$, then parties holding linear shares of $\text{sk}_i \cdot Z$ (for each bit sk_i of the secret key) can locally compute their share of $\text{sk}_i \cdot C_Z(x)$ as $\text{IP}(\langle \text{sk}_i \cdot Z \rangle_\sigma, x)$. While more involved HSS-based techniques have been developed to handle broader constraints (NC¹ [CMPR23] then P/poly [ILL24, MORS25]), they fail to achieve constraint privacy. Our new techniques achieve expressive constraints while preserving constraint privacy. We refer to fig. 1 for a more detailed overview of the CMPR template.

Fig. 1: The CMPR template for (privately) constrained PRFs from HSS. The template uses two HSS schemes: **HSS** should have “offline-online features”⁸(meaning the first party’s input share C_0 can be sampled in an offline phase ahead of the input C) and should support “dual constraints”, while **HSS** should follow the BGI template and support Restricted-Multiplication Straight-line (RMS, definition 7) programs.



2.2 Our Solution: Authenticating Powers of Shares

Recall from the previous section that our goal is to have the parties generate shares of $\mathbf{sk} \cdot C(x)$, where C is the private constraint and x is the public input. We take an “HSS approach” to this task and view it as homomorphically evaluating the public function $f : \alpha \mapsto \mathcal{U}(\alpha, x)$ (where $\mathcal{U}(\cdot, x)$ is a universal circuit, with the input x hardcoded) on shares of a secret value α , which encodes the constraint C . We will see in the next subsection that if the constraint class $\mathcal{C}$ only contains functions (with the same input space $\mathcal{X}$) of polynomial degree and Waring rank, then so does the dual class $\left\{ \begin{array}{l} f_x : \mathcal{C} \rightarrow \{0, 1\} \\ \alpha \mapsto \mathcal{U}(\alpha, x) \end{array} \mid x \in \mathcal{X} \right\}$, for every fixed x . Therefore we consider in this section the problem of generating (computational) shares of a secret input α which support homomorphic evaluation of any public function f with polynomially bounded degree and Waring rank.

We now demonstrate how two parties can locally compute shares of $\mathbf{sk} \cdot f(\alpha_i)$ provided they hold:

1. shares of each input bit (*i.e.* $\langle \alpha_i \rangle_\sigma$),
2. authenticated shares of each input bit (*i.e.* $\langle \mathbf{sk} \cdot \alpha_i \rangle_\sigma$),
3. encryptions of the first party’s input shares (*i.e.* $\text{Enc}_{\mathbf{sk}}(\langle \alpha_i \rangle_0)$)
4. encryptions of the first party’s authenticated input shares (*i.e.* $\text{Enc}_{\mathbf{sk}}(\langle \mathbf{sk} \cdot \alpha_i \rangle_0)$)
5. what we call “*authenticated encryptions*”⁹ of the first party’s input shares (*i.e.*, the ciphertext $\text{Enc}_{\mathbf{sk}}(\mathbf{sk} \cdot \langle \alpha_i \rangle_0)$).

Circuit Randomisation Step. Our goal is to express the function f in a form that allows efficient computation over secret shares. A degree- d polynomial f over $\{0, 1\}^m$ is said to have *Waring rank*¹⁰ at most r if there exist affine functions $f_1, \dots, f_r$, scalars $c_1, \dots, c_r$, and degrees $d_1, \dots, d_r \leq d$ such that $f(\alpha) = \sum_{i=1}^r c_i \cdot [f_i(\alpha)]^{d_i}$. We can use such a decomposition to simplify the problem of computing shares of $f(\alpha)$ (and $\mathbf{sk} \cdot f(\alpha)$):

$$\begin{aligned}
 f(\alpha) &= \sum_{i=1}^r c_i \cdot [f_i(\alpha)]^{d_i} && \text{(by definition of Waring rank)} \\
 &= \sum_{i=1}^r c_i \cdot \left[\underbrace{f'_i(\langle \alpha \rangle_1)}_{:= f_i(\langle \alpha \rangle_1) - f_i(0, \dots, 0)} + f_i(-\langle \alpha \rangle_0) \right]^{d_i} && \text{(by affinity of the } f_i) \\
 &= \sum_{i=1}^r \sum_{j=0}^{d_i} c_i \binom{d_i}{j} [f'_i(\langle \alpha \rangle_1)]^j \cdot [f_i(-\langle \alpha \rangle_0)]^{d_i-j} && \text{(by the binomial theorem)}
 \end{aligned}$$

Thus, computing subtractive shares of $f(\alpha)$ (resp. $\mathbf{sk} \cdot f(\alpha)$) reduces to computing subtractive shares of $[L(\langle \alpha \rangle_1)]^j \cdot [A(\langle \alpha \rangle_0)]^i$ (resp. $\mathbf{sk} \cdot [L(\langle \alpha \rangle_1)]^j \cdot [A(\langle \alpha \rangle_0)]^i$), where L is linear and A is affine.¹¹

⁸ Formalised in [CMPR23] as “staged-HSS”, or alternatively (and mostly equivalently) in [MORS25] as “offline-online HSS”.

⁹ Enc itself is just a (DDLog-friendly) encryption scheme, and does not need to be authenticated. The motivation for this terminology is that $X \mapsto \text{Enc}_{\mathbf{sk}}(\mathbf{sk} \cdot X)$ can be seen as an encryption scheme which internally uses $\mathbf{sk}$ to authenticate its input (in contrast with $\text{Enc}_{\mathbf{sk}}(\langle \mathbf{sk} \cdot \alpha_i \rangle)$, which is a “plain” encryption (and there, authentication happens within the share, not at an encryption level).

¹⁰ Formally, we use the mixed Waring rank notion here. Later in the paper, we prove that the standard and mixed Waring rank notions are polynomially related, meaning our results extend to both cases.

¹¹ Remember that all functions here take as input vectors of bits, e.g., $f(\alpha) = f(\alpha_1, \dots, \alpha_n)$, and $L(\langle \alpha \rangle_1) = L(\langle \alpha_1 \rangle_1, \dots, \langle \alpha_n \rangle_1)$, etc.

Authenticating Powers of linear combination of shares. The main technical contribution of this work is a new method that enables non-interactive evaluation of powers of authenticated shares. From a technical perspective, both our work and the concurrent work of [ILL24] are inspired by the techniques for non-interactive computation on shares presented in [MORS24] in the context of arithmetic garbling, but extend them in different directions.

Before detailing how the parties obtain shares for each term, recall that DDLog enables them to non-interactively compute shares of the product of a secret-shared value and an encrypted value. Specifically, if the parties hold $\langle X \rangle_\sigma$, $\langle \text{sk} \cdot X \rangle_\sigma$, and $\text{Enc}_{\text{sk}}(Y)$, they can use DDLog to derive $\langle X \cdot Y \rangle$ without additional interaction.

At a high level, our approach consists of the following steps: The first step is to obtain shares of $[L(\langle \alpha \rangle_1)]^j$ and $\text{sk} \cdot [L(\langle \alpha \rangle_1)]^j$, which the parties compute inductively.

Initialisation step ($j = 1$): This step requires computing shares of $L(\langle \alpha \rangle_1)$ and $\text{sk} \cdot L(\langle \alpha \rangle_1)$. The first part is straightforward, as the parties already hold tautological shares of $\langle \alpha \rangle_1$ —meaning that party 1 knows $\langle \alpha \rangle_1$ “in the clear,” and party 0 can simply set their own share to 0. Since subtractive shares are compatible with linear transformations:

$$\langle L(\langle \alpha \rangle_1) \rangle_\sigma \leftarrow \begin{cases} L(\langle \alpha \rangle_1) & \text{if } \sigma = 1 \\ 0 & \text{if } \sigma = 0 \end{cases}$$

To compute shares of $\text{sk} \cdot L(\langle \alpha \rangle_1)$, we observe that:

$$\begin{aligned} \text{sk} \cdot L(\langle \alpha \rangle_1) - \text{sk} \cdot L(\langle \alpha \rangle_0) &= \text{sk} \cdot L(\alpha) \\ &= L(\text{sk} \cdot \alpha) \\ &= L(\langle \text{sk} \cdot \alpha \rangle_1) - L(\langle \text{sk} \cdot \alpha \rangle_0) . \end{aligned} \tag{1}$$

This allows the parties to compute the desired shares as:

$$\langle \text{sk} \cdot L(\langle \alpha \rangle_1) \rangle_\sigma \leftarrow \begin{cases} L(\langle \text{sk} \cdot \alpha \rangle_1) & \text{if } \sigma = 1 \\ L(\langle \text{sk} \cdot \alpha \rangle_0) - \text{sk} \cdot L(\langle \alpha \rangle_0) & \text{if } \sigma = 0. \end{cases}$$

Induction step: Observe that the first part (computing shares of $[L(\langle \alpha \rangle_1)]^j$) follows the same tautological approach as before. For the second part (computing shares of $\text{sk} \cdot [L(\langle \alpha \rangle_1)]^j$), consider the identity:

$$\begin{aligned} \text{sk} \cdot [L(\langle \alpha \rangle_1)]^j &= (\text{sk} \cdot L(\langle \alpha \rangle_1)) \cdot [L(\langle \alpha \rangle_1)]^{j-1} \\ &= (\text{sk} \cdot L(\langle \alpha \rangle_0) + L(\langle \text{sk} \cdot \alpha \rangle_1) - L(\langle \text{sk} \cdot \alpha \rangle_0)) \cdot [L(\langle \alpha \rangle_1)]^{j-1} \quad (\text{by eq. (1)}) \\ &= \underbrace{\text{sk} \cdot [L(\langle \alpha \rangle_1)]^{j-1} \cdot L(\langle \alpha \rangle_0)}_{\text{Term 1}} + \underbrace{L(\langle \text{sk} \cdot \alpha \rangle_1) \cdot [L(\langle \alpha \rangle_1)]^{j-1}}_{\text{Term 2}} \\ &\quad - \underbrace{[L(\langle \alpha \rangle_1)]^{j-1} \cdot L(\langle \text{sk} \cdot \alpha \rangle_0)}_{\text{Term 3}} \end{aligned}$$

The parties obtain shares of each term as follows:

1. **First term.** The parties hold shares of $[L(\langle \alpha \rangle_1)]^{j-1}$ and $\text{sk} \cdot [L(\langle \alpha \rangle_1)]^{j-1}$ (by the induction hypothesis), as well as $\text{Enc}_{\text{sk}}(\text{sk} \cdot \langle \alpha \rangle_0)$. Using the linear homomorphic properties¹² of Enc , they compute $\text{Enc}_{\text{sk}}(\text{sk} \cdot L(\langle \alpha \rangle_0))$. By combining $\langle [L(\langle \alpha \rangle_1)]^{j-1} \rangle$, $\langle \text{sk} \cdot [L(\langle \alpha \rangle_1)]^{j-1} \rangle$, and the derived ciphertext with DDLog, they directly obtain shares of Term 1.

¹² It is important that the homomorphic evaluation be a deterministic procedure, ensuring both parties hold the *same* ciphertext.

2. **Second term.** Since party 1 already knows $\langle \mathbf{sk} \cdot \alpha \rangle_1$ and $\langle \alpha \rangle_1$, they can compute the second term locally. The parties then hold tautological shares:

$$\langle L(\langle \mathbf{sk} \cdot \alpha \rangle_1) \cdot [L(\langle \alpha \rangle_1)]^{j-1} \rangle_\sigma \leftarrow \begin{cases} L(\langle \mathbf{sk} \cdot \alpha \rangle_1) \cdot [L(\langle \alpha \rangle_1)]^{j-1} & \text{if } \sigma = 1 \\ 0 & \text{if } \sigma = 0 \end{cases}$$

3. **Third term.** Using the same approach as in the first term, the parties already hold shares of $[L(\langle \alpha \rangle_1)]^{j-1}$ and $\mathbf{sk} \cdot [L(\langle \alpha \rangle_1)]^{j-1}$ (from the induction hypothesis), as well as $\text{Enc}_{\mathbf{sk}}(\langle \mathbf{sk} \cdot \alpha \rangle_0)$. They can derive a common ciphertext $\text{Enc}_{\mathbf{sk}}(L(\langle \mathbf{sk} \cdot \alpha \rangle_0))$ via linear homomorphism and then use **DDLog** to compute shares of Term 3.

Sharing Mixed Term Products. Using linear homomorphism, the parties can derive the same ciphertext $\text{Enc}_{\mathbf{sk}}(\mathbf{sk} \cdot A(\langle \alpha \rangle_0))$. By leveraging their shares of $[L(\langle \alpha \rangle_1)]^j$ and $\mathbf{sk} \cdot [L(\langle \alpha \rangle_1)]^j$ (as computed in the previous steps), the parties can recursively apply **DDLog**—specifically, with $X_i \leftarrow [L(\langle \alpha \rangle_1)]^j \cdot [A(\langle \alpha \rangle_0)]^{i-1}$ and $Y \leftarrow \mathbf{sk} \cdot A(\langle \alpha \rangle_0)$ —to obtain shares of $\mathbf{sk} \cdot [L(\langle \alpha \rangle_1)]^j \cdot [A(\langle \alpha \rangle_0)]^i$.

2.3 Waring Rank

We now analyse the expressivity of the class of constraints that our technique can handle. As already mentioned, our technique allows us to evaluate (dual) constraints with polynomially bounded degree and Waring rank. The Waring rank of a polynomial f of degree d is the smallest number of linear polynomials whose $d_i < d$ powers sum to f . More formally, for some scalars c_i and linear polynomials L_i , it holds that

$$f(x) = \sum_{i=1}^r c_i (L_i(x))^{d_i},$$

where r is the Waring rank of f . For the special case of $d = 2$, the Waring rank equals the rank of the corresponding symmetric matrix representation of f .

Functions with polynomially bounded Waring rank include log-local functions, since they can be decomposed into sums of products of logarithmically many variables, and each such product can be handled by lemma 13. Another example is inner-product membership [BCM⁺24], which follows from the same techniques we describe below for puncturing.

Duality. Above, we promised that it is possible to dualise constraint functions with bounded degree and Waring rank, providing a kind of universal function for these constraints. Let $\alpha = \{(c_i, \{y_{i,j}\}_j)\}_i$ be the coefficients encoding a particular constraint, which has degree d and rank r .¹³ Then $C_\alpha(x)$ is evaluated as

$$C_\alpha(x) = \sum_{i=1}^r c_i \cdot (y_{i,0} + x_1 \cdot y_{i,1} + \cdots + x_n \cdot y_{i,n})^d \quad (2)$$

Now, we want to show that the dual constraint $\widehat{C}_x(\alpha) = C_\alpha(x)$ also has bounded Waring rank. To see this, first note that if the c_i were all constant then (2) is already a representation of $\widehat{C}_x(\alpha)$ with bounded Waring rank r . That is, we can simply swap the roles of x and y , leaving the inner product as a linear function of the input (which is now y). Handling the c_i as variables requires one extra step: multiplication between the bounded Waring rank functions c_i and $(y_{i,0} + \sum_j x_j \cdot y_{i,j})^d$. Luckily, this product can be shown to only blow up the Waring rank polynomially (see lemma 13 for more details).

Finally, the universal function need not reveal the Waring rank of the constraint, only an upper bound. If r is greater than the actual Waring rank of C_α , we can pad its representation with extra coefficients $c_i, y_{i,j} = 0$.

¹³ Here, we use non-mixed Waring rank to simplify the construction.

Applications of Waring Rank. Originally studied in invariant theory during the 19th century [IK99, Introduction], Waring rank has since found applications in several areas of theoretical computer science. In algebraic complexity theory, it characterises one of the simplest yet non-trivial complexity classes, namely homogeneous depth-3 diagonal circuits [DGI⁺24, Lan17, EGdOW18]. The problem of computing the matrix multiplication exponent ω , which determines the fastest known algorithm for matrix multiplication, has been shown to be closely related to Waring rank [CHI⁺18]. More recently, Pratt [Pra19] established an intriguing connection between algebraic algorithms for graph problems and upper bounds on the Waring rank of certain polynomials. Our work introduces a new connection between cryptography and Waring rank, showing how this algebraic notion captures a class of constraints for pCPRFs.

Waring Rank of Puncturing. Note that the puncturing constraint can be expressed as a polynomial with polynomially bounded degree and polynomially bounded Waring rank. Intuitively, $C_{x^*}(x) = 1$ (remember we want the constraint to be 1 for the punctured point) iff all the bits of x and x^* are the same. Thus, we can define a polynomial of degree n (for n -bit inputs), call it A_{x^*} , that counts the number of common bits between x and x^* . The puncturing constraint can then be rewritten as a product of $(A_{x^*}(x) - i)$ for all $i = 0, \dots, n-1$, resulting in 0 iff the number of common bits is between 0 and n these terms. This polynomial can then be normalized to ensure that $C_x(x^*) \in \{0, 1\}$.

Rounding Rational Shares Normalising the puncturing polynomial requires division, so the coefficients c_i will not be rationals. This is typical for expressing functions in Waring rank form – the expressive power is not great unless you have a sufficiently large field. Fortunately, we can work over $\mathbb{Q}$, and convert to integer shares as a final step. That is, if $y_0, y_1 \in \mathbb{Q}$ are rational shares of an integer $y_1 - y_0 = y$, then $\lfloor y_0 \rfloor, \lfloor y_1 \rfloor$ will be integer shares of the same y . Therefore, as long as f outputs integers, we can tolerate fractional coefficients in its definition.

3 Preliminaries

3.1 Damgård-Jurik-ElGamal Cryptosystem

Definition 1 (Decision Composite Residuosity Assumption (DCR), [Pai99]). Let RSA.Gen be a polynomial-time algorithm which, on input a security parameter λ , outputs (N, p, q) where p and q are λ -bit primes and $N = pq$. Let λ be a security parameter. We say that the Decision Composite Residuosity (DCR) problem is hard relative to modulus-sampling algorithm RSA.Gen if

$$\left\{ (N, x) : \begin{array}{l} (N, p, q) \xleftarrow{\$} \text{RSA.Gen}(1^\lambda) \\ x \xleftarrow{\$} (\mathbb{Z}/N^2\mathbb{Z})^\times \end{array} \right\} \stackrel{c}{\approx} \left\{ (N, x^N \bmod N^2) : \begin{array}{l} (N, p, q) \xleftarrow{\$} \text{RSA.Gen}(1^\lambda) \\ x \xleftarrow{\$} (\mathbb{Z}/N^2\mathbb{Z})^\times \end{array} \right\}.$$

Theorem 2. Assuming DCR, Damgård–Jurik–ElGamal encryption fig. 2 is a public key encryption scheme satisfying correctness and KDM security for affine functions of the key. Specifically, the following properties hold:

Correctness: $\text{DJE.Dec}_{\text{sk}}(\text{DJE.Enc}_{\text{pk}}(x)) = x$, for any (sk, pk) in the support of DJE.KeyGen , and any $x \in \mathbb{Z}/N^\zeta\mathbb{Z}$.

KDM Security: For all p.p.t. adversaries Adv , the oracles $\mathcal{O}_{\text{sk}, \text{pk}, R}^{\text{KDM}}$ and $\mathcal{O}_{\text{sk}, \text{pk}, \$}^{\text{KDM}}$ are indistinguishable in the following experiment

$$\begin{array}{c} (\text{sk}, \text{pk}) \xleftarrow{\$} \text{DJE.KeyGen}(1^\lambda) \\ \text{output } \text{Adv}_{\text{sk}, \text{pk}, R/\$}^{\text{KDM}}(\text{pk}) \end{array}$$

where these oracles are defined as

$\mathcal{O}_{\text{sk}, \text{pk}, R}^{\text{KDM}}(x, y):$ $(k, N) \leftarrow \text{sk}$ $z \leftarrow x \cdot k + y$ return $\text{DJE.Enc}_{\text{pk}}(z)$	$\mathcal{O}_{\text{sk}, \text{pk}, S}^{\text{KDM}}(x, y):$ $(k, N) \leftarrow \text{sk}$ $z \xleftarrow{S} \mathbb{Z}/N^\zeta \mathbb{Z}$ return $\text{DJE.Enc}_{\text{pk}}(z)$
--	--

Proof. This is very similar to many previous encryption schemes used in HSS and proven KDM secure in that context. See, e.g., the Damgård–Jurik instantiation of the NIDLS framework [ADOS22]), though details differ, as we do not require p and q to be safe primes. For completeness, we present a full proof in appendix A.2. $\square$

DJE Damgård–Jurik–ElGamal Cryptosystem

Requires:

- $\zeta \geq 1$ is a parameter defining the plaintext size.
- Group isomorphism $\exp: (\mathbb{Z}/N^\zeta \mathbb{Z})^+ \rightarrow 1 + N(\mathbb{Z}/N^{\zeta+1} \mathbb{Z})$ and its inverse $\log: 1 + N(\mathbb{Z}/N^{\zeta+1} \mathbb{Z}) \rightarrow (\mathbb{Z}/N^\zeta \mathbb{Z})^+$, as defined as in [RS21]:

$$\exp(x) = \sum_{k=0}^{\zeta} \frac{(Nx)^k}{k!} \quad \text{and} \quad \log(1 + Nx) = \sum_{k=1}^{\zeta} \frac{(-N)^{k-1} x^k}{k}$$

DJE.KeyGen(1^λ):

1. Sample $(N, p, q) \xleftarrow{S} \text{RSA.Gen}(1^\lambda)$ ^a
2. Sample $k \xleftarrow{S} [0, N)$
3. Sample $g \xleftarrow{S} (\mathbb{Z}/N^{\zeta+1} \mathbb{Z})^\times$
4. Compute $h \leftarrow g^{-k}$
5. Output $(\text{sk} = (k, N), \text{pk} = (g, h, N))$

DJE.Enc_{pk}(x):

1. Parse $\text{pk} = (g, h, N)$
2. Sample $r \xleftarrow{S} [0, N)$
3. Compute $c_0 \leftarrow g^r$
4. Compute $c_1 \leftarrow h^r \cdot \exp(x)$
5. Output $c = (c_0, c_1)$

^a Note that p and q are unused, so N could instead be a CRS.

DJE.Dec_{sk}($c = (c_0, c_1)$):

1. Parse $\text{sk} = (k, N)$
2. Assert $c_0^k \cdot c_1 \equiv 1 \pmod{N}$
3. Output $x \leftarrow \log(c_0^k \cdot c_1)$

Fig. 2: The Damgård–Jurik–ElGamal cryptosystem.

3.2 Distributed Discrete Logarithm

DDLOG Damgård-Jurik Distance Function [RS21]

$\text{DDLog}_N(h \in \mathbb{Z}/N^{\zeta+1}\mathbb{Z})$:

Compute and output $z \leftarrow \log\left(\frac{h}{h \bmod N}\right) \in \mathbb{Z}/N^\zeta\mathbb{Z}$

Fig. 3: [RS21]’s distributed discrete logarithm for the Damgård-Jurik cryptosystem [DJ01].

Lemma 3 (Distributed Decryption). *If we have shares over $\mathbb{Z}$ of $\langle x \rangle_1 - \langle x \rangle_0 = x$ and $\langle k \cdot x \rangle_1 - \langle k \cdot x \rangle_0 = k \cdot x$, then*

$$\text{DDLog}_N(c_0^{\langle k \cdot x \rangle_1} c_1^{\langle x \rangle_1}) - \text{DDLog}_N(c_0^{\langle k \cdot x \rangle_0} c_1^{\langle x \rangle_0}) \equiv x \cdot y \pmod{N^\zeta}$$

always holds, for every choice of plaintext size $\zeta \geq 1$, key pair $(\text{sk} = (k, N), \text{pk}) \in \text{Supp}(\text{DJE.KeyGen}(1^\lambda))$, plaintext $y \in \mathbb{Z}/N^\zeta\mathbb{Z}$, ciphertext $(c_0, c_1) \in \text{Supp}(\text{DJE.Enc}_{\text{pk}}(y))$, and scalar $x \in \mathbb{Z}/N^\zeta\mathbb{Z}$.

Proof. Taking the ratio of the inputs to DDLog, we get

$$\frac{c_0^{\langle k \cdot x \rangle_1} c_1^{\langle x \rangle_1}}{c_0^{\langle k \cdot x \rangle_0} c_1^{\langle x \rangle_0}} = c_0^{k \cdot x} c_1^x = \exp(\text{DJE.Dec}_{\text{sk}}(c_0, c_1))^x = \exp(x \cdot y).$$

The second and third equalities are from the definition of DJE.Dec and the correctness of DJE, respectively. The inputs to the DDLogs are therefore multiplicative shares of $\exp(x \cdot y)$, which, by [RS21, Theorem 18], makes the outputs of the DDLogs additive shares over $\mathbb{Z}/N^\zeta\mathbb{Z}$ of $x \cdot y$. $\square$

Lemma 4 (Adapted from [RS21, Lemma 19]). *For all moduli $M > 1$ and all modulo M shares $\langle x \rangle_0, \langle x \rangle_1 \in \mathbb{Z}/M\mathbb{Z}$ of some $x \in \mathbb{Z}$, we have*

$$\Pr_{r \leftarrow \mathbb{Z}/M\mathbb{Z}} \left[(\langle x \rangle_1 + r) \bmod M - (\langle x \rangle_0 + r) \bmod M = x \right] = \max\left(1 - \frac{|x|}{N^\zeta}, 0\right).$$

3.3 Privately Constrained Pseudorandom Function

We use the simulation-based definition of Peikert-Shiehian [PS18], which simultaneously captures privacy of the constraining function, pseudorandomness on unauthorised inputs, and correctness of constrained evaluation on authorised inputs.

Definition 5 (Privately Constrained Pseudorandom Function). *A privately constrained PRF (pCPRF) with input space $\{0, 1\}^n$ and output space $\{0, 1\}^\mu$ and supporting as constraints a class of circuits $\mathcal{C}$ (where each circuit in $\mathcal{C}$ is from $\{0, 1\}^n$ to $\{0, 1\}$) is a tuple of algorithms $\text{PCPRF} = (\text{PCPRF.KeyGen}, \text{PCPRF.Eval}, \text{PCPRF.Constrain}, \text{PCPRF.CEval})$ with the following syntax and properties:*

- $\text{PCPRF.KeyGen}(1^\lambda) \rightarrow \text{msk}$: On input a security parameter 1^λ , the key generation algorithm KeyGen produces a master secret key msk.
- $\text{PCPRF.Eval}(1^\lambda, \text{msk}, x) \rightarrow y$: On input a security parameter 1^λ , a master secret key msk, and an input $x \in \{0, 1\}^n$, the evaluation algorithm Eval output an element $y \in \{0, 1\}^\mu$.

- $\text{PCPRF.Constrain}(1^\lambda, \text{msk}, C) \rightarrow \text{ck}$: On input a security parameter 1^λ , a master secret key msk , and a constraining circuit $C \in \mathcal{C}$, the constraining algorithm **Constrain** outputs a constrained key ck .
- $\text{PCPRF.CEval}(1^\lambda, \text{ck}, x) \rightarrow y$: On input a security parameter 1^λ , a constrained secret key ck , and an input $x \in \{0, 1\}^n$, the constrained evaluation algorithm **CEval** output an element $y \in \{0, 1\}^\mu$.
- **Single-key selective PCPRF security.** A privately constrained PRF is single-key selectively secure if for any p.p.t. adversary $\mathcal{A}$ (which we require to never repeat a query) and any $m \in \{0, 1\}^\lambda$ there exists a p.p.t. simulator $\mathcal{S}$ such that:

$$\left\{ \text{Real}_{\mathcal{A}}(1^\lambda, 1^m) \right\}_{\lambda \in \mathbb{N}} \stackrel{c}{\approx} \left\{ \text{Ideal}_{\mathcal{A}, \mathcal{S}}(1^\lambda, 1^m) \right\}_{\lambda \in \mathbb{N}}$$

where **Real** and **Ideal** are the respective views of $\mathcal{A}$ in the experiments of fig. 4.

Experiment Privately Constrained PRF Security	
$\text{Real}_{\mathcal{A}}(1^\lambda, 1^m)$ $C \xleftarrow{\$} \mathcal{A}(1^\lambda, 1^m)$ $\text{msk} \xleftarrow{\$} \text{KeyGen}(1^\lambda)$ $\text{ck} \xleftarrow{\$} \text{Constrain}(1^\lambda, \text{msk}, C)$ Send ck to $\mathcal{A}$ Repeat until $\mathcal{A}$ halts: Wait to receive x from $\mathcal{A}$ $y \xleftarrow{\$} \text{Eval}(1^\lambda, \text{msk}, x)$ Send y to $\mathcal{A}$	$\text{Ideal}_{\mathcal{A}, \mathcal{S}}(1^\lambda, 1^m)$ $C \xleftarrow{\$} \mathcal{A}(1^\lambda, 1^m)$ $\text{ck} \xleftarrow{\$} \mathcal{S}(1^\lambda, 1^m)$ Send ck to $\mathcal{A}$ Repeat until $\mathcal{A}$ halts: Wait to receive x from $\mathcal{A}$ If $C(x) = 0$: $y \xleftarrow{\$} \text{CEval}(1^\lambda, \text{ck}, x)$ If $C(x) = 1$: $y \xleftarrow{\$} \{0, 1\}^\mu$ Send y to $\mathcal{A}$

Fig. 4: Real and Ideal worlds of PCPRF security (definition 5).

4 Share Conversion Procedures

The heart of any¹⁴ homomorphic secret sharing scheme is its evaluation algorithm, rather than its sharing or reconstruction algorithms. The technical core is almost always a “share conversion” procedure, allowing each party to locally convert its input share into an output share. Strictly speaking, “instantiating” the CMPR template would require providing a construction of staged HSS [CMPR23, definition 6] supporting an appropriately expressive class of homomorphic computation. With the notations of fig. 1, this is the first HSS scheme **HSS**. In this work, instead of relying on the staged HSS formalism, we make the choice to isolate the “share conversion procedure” (homomorphically evaluating functions with polynomially bounded degree and Waring rank) in order to better emphasise our core technical contribution.

¹⁴ This statement is only intended as a high-level comment on known constructions.

In this section, we detail three “homomorphic share conversion” algorithms (one is our own, and two are from prior works) based on distributed discrete logarithms, which allow two parties holding appropriately encoded representation (namely, shares, authenticated shares, encryptions) of some inputs to compute shares of some function of these inputs.

1. **Multiplication.** Section 4.1 (and specifically fig. 5) recalls from [BGI16] (with the “DCR variants” of [OSY21, RS21]) how to compute $\langle x \cdot y \rangle$ given $\langle x \rangle$, $\langle k \cdot x \rangle$, $\text{Enc}_k(y)$.
2. **Bounded Waring Rank.** Section 4.2 (and specifically fig. 6) is the main technical contribution of this paper and shows how to compute $\langle (1, \text{sk}) \cdot f(x) \rangle$, where f is any (public) polynomial with bounded¹⁵ degree and Waring rank, given $\langle x_i \rangle$, $\langle \text{sk} \cdot x_i \rangle$, $\text{Enc}_{\text{sk}}(\langle x_i \rangle_0)$, $\text{Enc}_{\text{sk}}(\langle \text{sk} \cdot x_i \rangle_0)$, $\text{Enc}_{\text{sk}}(\text{sk} \cdot \langle x_i \rangle_0)$.
3. **RMS Programs (definition 7) with Leading Multiplication.** Section 4.3 (and specifically fig. 7) recalls from [CMPR23] how to compute $\langle X \cdot \text{RMS}(y_1, \dots, y_l) \rangle$ for any RMS program RMS, given $\langle X \rangle$, $\langle \text{sk} \cdot X \rangle$, $\text{Enc}_{\text{sk}}(y_i)$, and $\text{Enc}_{\text{sk}}(\text{sk} \cdot y_i)$.

4.1 Homomorphic Share Conversion: Multiplication

Starting with the seminal work of Boyle, Gilboa, and Ishai [BGI16], HSS schemes tend to proceed in a gate-by-gate fashion. Algorithm `HSS.Share` produces “input shares” (typically authenticated additive shares of the inputs, and linearly homomorphic encryptions of the inputs), and algorithm `HSS.Eval` then maintains “memory shares” (typically authenticated additive shares) of each intermediary wire label of the circuit. The technical core of that construction (and what characterises the “BGI template” for HSS) is a procedure combining an input share of x with a memory share of y in order to compute a memory share of xy . In other words, given $\langle x \rangle_\sigma$, $\langle k \cdot x \rangle_\sigma$, $\text{Enc}_k(y)$ the share conversion procedure (for a single multiplication) computes $\langle x \cdot y \rangle_\sigma$. We recall the procedure in fig. 5, and recall in lemma 6 its correctness property. For technical reasons (tied to making this output homogeneous with a uniformly sampled memory share) the shares are offset by the same random value.

Algorithm Homomorphic Share Conversion: Multiplication

Mult(sh, auth-sh, ct, r) : // Given $\langle x \rangle$, $\langle k \cdot x \rangle$, $\text{Enc}_k(y)$, $r \in [N^\zeta]$ compute $\langle x \cdot y \rangle$ (randomised by r)

1. Parse ct as $\text{ct} = (\text{ct}_0, \text{ct}_1)$
2. Output $\text{DDLog}(\text{ct}_0^{\text{auth-sh}} \cdot \text{ct}_1^{\text{sh}}) + r \bmod N^\zeta$

Fig. 5: Homomorphic multiplication of secret shares, from DDLog “à la ElGamal”, adapted from [BGI16, OSY21].

Lemma 6 (Correctness of share conversion procedure for multiplications). *Let $\lambda \in \mathbb{N}^*$ be a security parameter and let $B = \lambda^{\mathcal{O}(1)}$. For all $(x, y) \in [0, B]^2$ such that $xy \leq B$ and for all $\text{ct} \in \text{Supp}(\text{DJE}.\text{Enc}_{\text{pk}}(y))$,*

$$\Pr \left[y_1 - y_0 = x \cdot y : (y_0, y_1) \xleftarrow{\$} \text{Sam}(1^\lambda, x, y, \text{ct}) \right] \geq 1 - B/N^{\zeta-1}$$

where the sampler **Sam** is defined as follows (where **Mult** is the deterministic algorithm of fig. 5 and **DJE** is the cryptosystem of fig. 3):

Sam($1^\lambda, x, y, \text{ct}$):

¹⁵ The bound can be an arbitrary polynomial in the security parameter.

1. $((k, N), \text{pk}) \xleftarrow{\$} \text{DJE.KeyGen}(1^\lambda)$
2. $\text{sh}_0 \xleftarrow{\$} [0, 2^\lambda B)$
3. $\text{sh}_1 \leftarrow x + \text{sh}_0$
4. $\text{sh}_0^{\text{auth}} \xleftarrow{\$} [0, N \cdot 2^\lambda B)$
5. $\text{sh}_1^{\text{auth}} \leftarrow k \cdot x + \text{sh}_0^{\text{auth}}$
6. $r \xleftarrow{\$} \mathbb{Z}/N^\zeta \mathbb{Z}$
7. For $\sigma \in \{0, 1\}$, $y_\sigma \leftarrow \text{Mult}(\text{sh}_\sigma, \text{sh}_\sigma^{\text{auth}}, \text{ct}, r)$
8. Output (y_0, y_1)

Lemma 6 follows immediately from lemma 18.

4.2 Homomorphic Share Conversion: Bounded Waring Rank

As we describe in section 2.2, our main contribution is a share conversion algorithm to transform input shares of x into memory shares of $f(x)$, for any function f with polynomially bounded degree and Waring rank. While memory shares are as in section 4.1 (namely, shares $\langle f(x) \rangle_\sigma$ and authenticated shares $\langle \text{sk} \cdot f(x) \rangle_\sigma$), the input shares here are more involved: they are comprised of

- shares $\langle x \rangle_\sigma$,
- authenticated shares $\langle \text{sk} \cdot x \rangle_\sigma$,
- encryptions of the 0-shares $\text{Enc}_{\text{sk}}(\langle x \rangle_0)$,
- encryptions of the authenticated 0-shares $\text{Enc}_{\text{sk}}(\langle \text{sk} \cdot x \rangle_0)$,
- authenticated encryptions of the (unauthenticated) 0-shares $\text{Enc}_{\text{sk}}(\text{sk} \cdot \langle x \rangle_0)$.

In addition the input shares contain some input-independent information: encryptions of 1 and of the secret key sk . While proving that such input shares are indeed secret shares is beyond the scope of this section (recall that a “share conversion” procedure is only meant to capture the correctness of an HSS evaluation algorithm), we establish this fact as part of the proof in appendix B.2.1.

Section 2.2 already presented all the steps of the share conversion algorithm, but in reverse, showing how generating shares of $f(x)$ and $\text{sk} \cdot f(x)$ could be iteratively reduced to simpler and simpler tasks. The formal procedure of fig. 6 proceeds the other way: showing how to build up to shares of $f(x)$ and $\text{sk} \cdot f(x)$ by progressively building more and more involved shares, while starting from scratch.

Algorithm Homomorphic Share Conversion: Bounded Waring Rank

// Compute $\langle f(x_1, \dots, x_n) \rangle_\sigma$ and $\langle k \cdot f(x_1, \dots, x_n) \rangle_\sigma$.

Requires:

1. r and d are fixed bounds on the Waring rank and the degree of functions which can be evaluated homomorphically
2. DJE is the cryptosystem of fig. 2
3. Mult is the algorithm of fig. 5
4. HEval is the linearly homomorphic evaluation algorithm for the Damgård-Jurik-ElGamal encryption scheme

$\text{ShareConv}(\sigma,$ // The party index
 $k,$ // The key (only needed for $\sigma = 0$)
 $(\alpha_i, f_i, d_i)_{i \in [r]},$ // $f = \sum_{i=1}^r \alpha_i f_i^{d_i}$
 $(\text{sh}_{\sigma,i})_{i \in [m]},$ // $\langle x_i \rangle_\sigma$ (λ -bit shares over $\mathbb{Z}$)
 $(\text{sh}_{\sigma,i}^{\text{auth}})_{i \in [m]},$ // $\langle k \cdot x_i \rangle_\sigma$ ($(\lambda \cdot \log N)$ -bit shares over $\mathbb{Z}$)
 $(c_i = (c_{i,0}, c_{i,1}))_{i \in [m]},$ // An encryption under (k, N) of $\text{sh}_{0,i}$
 $(c_i^{\text{auth}} = (c_{i,0}^{\text{auth}}, c_{i,1}^{\text{auth}}))_{i \in [m]},$ // An encryption of $\text{sh}_{0,i}^{\text{auth}}$
 $(c_i^{\text{ext-auth}} = (c_{i,0}^{\text{ext-auth}},$ // Encryption of $k \cdot \text{sh}_{0,i}$
 $c_{i,1}^{\text{ext-auth}}))_{i \in [m]},$
 $(c_{\text{cst}}, c_{\text{cst}}^{\text{auth}}),$ // Encryptions of 1 and k
 $\text{crs}) :$ // A random string in $(\mathbb{Z}/N^\zeta\mathbb{Z})^{4 \cdot rd}$

1. For $i \in [r]$, $f'_i \leftarrow f_i - f_i(0, \dots, 0)$ // Linear part of affine function f_i

2. Parse crs as $\text{crs} = (\text{crs}_{i,j,b})_{i \in [r], j \in [d], b \in [4]} \in [N^\zeta]^{4rd}$

3. Initialise table TAB_σ , indexed by $[r] \times [d] \times [d]$
// $\text{TAB}_\sigma[i][j][\ell]$ will be used to store $\langle [f'_i(\langle x \rangle_1)]^j \cdot [f_i(-\langle x \rangle_0)]^\ell \rangle_\sigma$

4. Initialise table $\text{TAB}_\sigma^{\text{auth}}$, indexed by $[r] \times [d] \times [d]$
// $\text{TAB}_\sigma^{\text{auth}}[i][j][\ell]$ will be used to store $\langle k \cdot [f'_i(\langle x \rangle_1)]^j \cdot [f_i(-\langle x \rangle_0)]^\ell \rangle_\sigma$

5. **Compute the $\text{Enc}(k \cdot f'_i(\langle x \rangle_0))$ (for $i \in [r]$).**

$$\text{ct}'_i \leftarrow \text{DJE.HEval}(f'_i, c_1^{\text{ext-auth}}, \dots, c_n^{\text{ext-auth}})$$

6. **Compute the $\text{Enc}(f'_i(\langle k \cdot x \rangle_0))$ (for $i \in [r]$).**

$$\text{ct}''_i \leftarrow \text{DJE.HEval}(f'_i, c_1^{\text{auth}}, \dots, c_n^{\text{auth}})$$

7. **Compute the $\text{Enc}(f_i(-\langle x \rangle_0))$ (for $i \in [r]$).**

$$\text{ct}'''_i \leftarrow \text{DJE.HEval}(g_i, c_1, \dots, c_n, c_{\text{cst}})$$

where $g_i: (X_1, \dots, X_n, X_{m+1}) \mapsto f'_i(-X_1, \dots, -X_n) + f_i(0, \dots, 0) \cdot X_{m+1}$

8. **Compute the $\text{Enc}(k \cdot f_i(-\langle x \rangle_0))$ (for $i \in [r]$).**

$$\text{ct}''''_i \leftarrow \text{DJE.HEval}(g_i, c_1^{\text{ext-auth}}, \dots, c_n^{\text{ext-auth}}, c_{\text{cst}}^{\text{auth}})$$

9. **Compute the $\langle k \cdot [f'_i(\langle x \rangle_1)]^j \rangle_\sigma$ (for $1 \leq j \leq i \leq r$).**

For $i = 1, \dots, r$ do (in parallel):

(a) For $j = 1, \dots, d_i$ do (in parallel):

$$\text{TAB}_\sigma[i][j][0] \leftarrow \begin{cases} [f'_i(\langle x_1 \rangle_1, \dots, \langle x_n \rangle_1)]^j & \text{if } \sigma = 1 \\ 0 & \text{if } \sigma = 0 \end{cases}$$

(b) Set $\text{TAB}_\sigma^{\text{auth}}[i][1][0]$ to

$$\begin{cases} f'_i(\text{sh}_{1,1}^{\text{auth}}, \dots, \text{sh}_{1,m}^{\text{auth}}) & \text{if } \sigma = 1 \\ f'_i(\text{sh}_{0,1}^{\text{auth}}, \dots, \text{sh}_{0,m}^{\text{auth}}) - k \cdot f'_i(\text{sh}_{0,1}, \dots, \text{sh}_{0,m}) & \text{if } \sigma = 0 \end{cases}$$

(c) For $j = 2, \dots, d_i$ (sequentially), set $\text{TAB}_\sigma^{\text{auth}}[i][j][0]$ to

$$\begin{cases} f'_i(\text{sh}_{1,1}^{\text{auth}}, \dots, \text{sh}_{1,m}^{\text{auth}}) \cdot [f'_i(\text{sh}_{1,1}, \dots, \text{sh}_{1,m})]^{j-1} \\ + \text{Mult}(\text{TAB}_1[i][j-1][0], \text{TAB}_1^{\text{auth}}[i][j-1][0], \text{ct}'_i, \text{crs}_{i,j,1}) & \text{if } \sigma = 1 \\ - \text{Mult}(\text{TAB}_1[i][j-1][0], \text{TAB}_1^{\text{auth}}[i][j-1][0], \text{ct}''_i, \text{crs}_{i,j,2}) \\ \\ \text{Mult}(\text{TAB}_0[i][j-1][0], \text{TAB}_0^{\text{auth}}[i][j-1][0], \text{ct}'_i, \text{crs}_{i,j,1}) \\ - \text{Mult}(\text{TAB}_0[i][j-1][0], \text{TAB}_0^{\text{auth}}[i][j-1][0], \text{ct}''_i, \text{crs}_{i,j,2}) & \text{if } \sigma = 0 \end{cases}$$

10. Compute the $\langle k \cdot [f'_i(\langle x \rangle_1)]^j \cdot [f_i(-\langle x \rangle_0)]^\ell \rangle_\sigma$ (for $0 \leq j < i \leq r$ and $\ell \in [d_i]$).

For $i = 1, \dots, r$ do (in parallel):

(a) Set $\text{TAB}_\sigma[i][0][d_i]$ to

$$\begin{cases} 0 & \text{if } \sigma = 1 \\ [f_i(-\text{sh}_{0,1}, \dots, -\text{sh}_{0,m})]^{d_i} & \text{if } \sigma = 0 \end{cases}$$

(b) Set $\text{TAB}_\sigma^{\text{auth}}[i][0][d_i]$ to

$$\begin{cases} 0 & \text{if } \sigma = 1 \\ k \cdot [f_i(-\text{sh}_{0,1}, \dots, -\text{sh}_{0,m})]^{d_i} & \text{if } \sigma = 0 \end{cases}$$

(c) For $j = 1, \dots, d_i$ do (in parallel):

For $\ell = 1, \dots, d_i - j$ do (sequentially):

$$\begin{aligned} \text{TAB}_\sigma[i][j][\ell] &\leftarrow \text{Mult}(\text{TAB}_\sigma[i][j][\ell-1], \\ &\quad \text{TAB}_\sigma^{\text{auth}}[i][j][\ell-1], \\ &\quad \text{ct}'''_i, \text{crs}_{i,j,3}) \\ \text{TAB}_\sigma^{\text{auth}}[i][j][\ell] &\leftarrow \text{Mult}(\text{TAB}_\sigma[i][j][\ell-1], \\ &\quad \text{TAB}_\sigma^{\text{auth}}[i][j][\ell-1], \\ &\quad \text{ct}''''_i, \text{crs}_{i,j,4}) \end{aligned}$$

11. Compute $\langle f(x) \rangle_\sigma$ and $\langle k \cdot f(x) \rangle_\sigma$.

$$\begin{aligned} y_\sigma &\leftarrow \sum_{i=1}^r \sum_{j=0}^{d_i} \alpha_i \cdot \binom{d_i}{j} \cdot (-1)^{d_i-j} \cdot \text{TAB}_\sigma[i][j][d_i-j] \\ y_\sigma^{\text{auth}} &\leftarrow \sum_{i=1}^r \sum_{j=0}^{d_i} \alpha_i \cdot \binom{d_i}{j} \cdot (-1)^{d_i-j} \cdot \text{TAB}_\sigma^{\text{auth}}[i][j][d_i-j] \end{aligned}$$

12. Output $(\lfloor y_\sigma \rfloor, \lfloor y_\sigma^{\text{auth}} \rfloor)$

Fig. 6: Share conversion procedure for homomorphically evaluating functions of bounded Waring rank.

Core Lemma 1 (Correctness of share conversion procedure for bounded Waring rank). *Let*

$\lambda \in \mathbb{N}^*$ be a security parameter, let $(\lambda_{\text{DJE}}, \zeta)$ be parameters for the Damgård-Jurik-ElGamal cryptosystem, let r, d be fixed polynomials in the security parameter, and let $f = \sum_{i=1}^r \alpha_i \cdot f_i^{d_i}$ be a integer-valued degree- d polynomial with Waring rank at most r (where $f_1, \dots, f_r$ are affine functions and $\alpha_1, \dots, \alpha_r \in \mathbb{Q}$), and consider the deterministic algorithm **ShareConv** of fig. 6 parameterised by Damgård-Jurik parameter ζ . Let β be a bound on the coefficients of the f_i . For any inputs $x_1, \dots, x_n \in [0, B]^m$,

$$\Pr \left[y_1 - y_0 = \text{sk} \cdot f(x_1, \dots, x_n) : (\text{sk}, y_0, y_1) \xleftarrow{\$} \text{Sam}(1^\lambda, (x_i)_{i=1}^m) \right] \geq 1 - \frac{4rd^2(nB2^\lambda\beta)^d}{(2^{\lambda_{\text{DJE}}-1})^{\zeta-1}}$$

when $f(x_1, \dots, x_n) \in \mathbb{Z}$, where the sampler **Sam** is defined as:

Sam $(1^\lambda, (x_i)_{i \in [m]}):$

1. $((k, N), \text{pk}) \xleftarrow{\$} \text{DJE.KeyGen}(1^\lambda)$
2. $\text{sh}_{0,i} \xleftarrow{\$} [0, 2^\lambda B]$
3. $\text{sh}_{1,i} \leftarrow x_i + \text{sh}_{0,i}$
4. $\text{sh}_{0,i}^{\text{auth}} \xleftarrow{\$} [0, N \cdot 2^\lambda B]$
5. $\text{sh}_{1,i}^{\text{auth}} \leftarrow k \cdot x_i + \text{sh}_{0,i}^{\text{auth}}$
6. $c_i \xleftarrow{\$} \text{DJE.Enc}_{\text{pk}}(\text{sh}_{0,i})$
7. $c_i^{\text{auth}} \xleftarrow{\$} \text{DJE.Enc}_{\text{pk}}(\text{sh}_{0,i}^{\text{auth}})$
8. $c_i^{\text{ext-auth}} \xleftarrow{\$} \text{DJE.Enc}_{\text{pk}}(\text{sk} \cdot \text{sh}_{0,i})$
9. $\text{crs} \xleftarrow{\$} (\mathbb{Z}/N^\zeta\mathbb{Z})^{4rd}$
10. For $\sigma \in \{0, 1\}$,
 $y_\sigma \leftarrow \text{ShareConv}(\sigma, (\alpha_i, f_i, d_i)_{i \in [r]},$
 $(\text{sh}_{\sigma,i})_{i \in [m]}, (\text{sh}_{\sigma,i}^{\text{auth}})_{i \in [m]},$
 $(c_i)_{i \in [m]}, (c_i^{\text{auth}})_{i \in [m]},$
 $(c_i^{\text{ext-auth}})_{i \in [m]}, \text{crs})$
11. Output (sk, y_0, y_1)

We refer to appendix B.1 for the proof of core lemma 1.

4.3 Homomorphic Share Conversion: RMS Programs with Leading Multiplication

Recall from section 2.1 that the final step of the CMPR template is to combine memory shares of $\text{sk} \cdot C(x)$ with input shares of a PRF key k_{PRF} in order to yield (memory) shares of $C(x) \cdot \text{PRF}_{k_{\text{PRF}}}(x)$. In this section, we describe this share conversion procedure, which is due to [CMPR23].

The main idea is to take the share conversion procedure underlying (the evaluation algorithm of) BGI-style HSS for RMS programs (definition 7), and to observe that if one loads inputs into memory using $\langle \text{sk} \cdot X \rangle_\sigma$ (for some scalar X) instead of $\langle \text{sk} \rangle_\sigma$, then the resulting memory share inherits the factor X . This leading factor can then be propagated through the gate-by-gate evaluation (observe that it is preserved through additions, and that multiplications only ever involves an input and a memory share, and the factor X is only present in the latter).

Definition 7 (Restricted-Multiplication Straight-line Programs, adapted from [BGI16]). A (single-output) *restricted-multiplication straight-line (RMS) program* is an arbitrary sequence of instructions, each matching one of the following four:

- *Loading an input into memory:* (“Load”, $\hat{y}_j, \hat{w}_i$) // Instructing to load input $\hat{w}_i$ into memory value/variable $\hat{y}_j$
- *Add values in memory:* (“Add”, $\hat{y}_k, \hat{y}_i, \hat{y}_j$) // Instructing to store the sum of memory values/variables $\hat{y}_i$ and $\hat{y}_j$ in memory value/variable $\hat{y}_k$
- *Multiply value in memory by an input value:* (“Multiply”, $\hat{y}_k, \hat{w}_i, \hat{y}_j$) // Instructing to store the product of input $\hat{w}_i$ and memory value/variable $\hat{y}_j$ in memory value/variable $\hat{y}_k$

- *Output value from memory:* $(\text{Output}, j, \hat{y}_i)$ // Instructing to output the memory value $\hat{y}_i$

We say a set of inputs is admissible to an RMS program with respect to bound B if all memory values remain bounded by B .

Algorithm Homomorphic Share Conversion: RMS Programs with Leading Multiplication, adapted from [CMPR23]

// Given $\langle X \rangle$, $\langle \text{sk} \cdot X \rangle$, $\text{Enc}_{\text{sk}}(y_i)$, and $\text{Enc}_{\text{sk}}(\text{sk} \cdot y_i)$, compute $\langle X \cdot \text{RMS}(y_1, \dots, y_1) \rangle$.

Requires: Mult is the algorithm of fig. 5. The scheme is additionally parameterised by parameters N, ζ .

ShareConvExtRMS(σ , // Party index
 pk , // Pk for the enc. underlying DDLog
 Π , // An RMS program
 sh_σ , // $\langle X \rangle_\sigma$
 $\text{sh}_\sigma^{\text{auth}}$, // $\langle \text{sk} \cdot X \rangle_\sigma$
 $(c_i)_{i \in [m]}$, // $\text{Enc}_{\text{sk}}(y_i)$ for $i \in [m]$
 $(c_i^{\text{auth}})_{i \in [m]}$, // $\text{Enc}_{\text{sk}}(\text{sk} \cdot y_i)$ for $i \in [m]$
 crs) // A $(M \cdot \zeta \cdot \log N)$ -bit string

1. Parse crs as $(\text{crs}_j)_{j \in [M]} \in (\mathbb{Z}/N^\zeta\mathbb{Z})^M$
2. Parse Π as a sequence of RMS instructions $\text{op}_1, \dots, \text{op}_M$.
3. Initialise a vector MemShares_σ indexed by $[M]$.
// Used to store in position ℓ the memory value resulting from op_ℓ .
4. Initialise a vector $\text{MemShares}_\sigma^{\text{auth}}$ indexed by $[M]$.
// Used to store in position ℓ the authenticated memory value resulting from op_ℓ .
5. For $\ell = 1, \dots, M$:
 - If op_ℓ is of the form (“Load”, $\hat{y}_j, \hat{w}_i$):

$$\begin{aligned} \text{MemShares}_\sigma[j] &\leftarrow \text{Mult}(\text{sh}_\sigma, \text{sh}_\sigma^{\text{auth}}, c_i, \text{crs}_\ell) \\ \text{MemShares}_\sigma^{\text{auth}}[j] &\leftarrow \text{Mult}(\text{sh}_\sigma, \text{sh}_\sigma^{\text{auth}}, c_i^{\text{auth}}, \text{crs}_\ell) \end{aligned}$$

- If op_ℓ is of the form (“Add”, $\hat{y}_q, \hat{y}_i, \hat{y}_j$):

$$\begin{aligned} \text{MemShares}_\sigma[q] &\leftarrow \text{MemShares}_\sigma[i] + \text{MemShares}_\sigma[j] \\ \text{MemShares}_\sigma^{\text{auth}}[q] &\leftarrow \text{MemShares}_\sigma^{\text{auth}}[i] + \text{MemShares}_\sigma^{\text{auth}}[j] \end{aligned}$$

- If op_ℓ is of the form (“Multiply”, $\hat{y}_q, \hat{w}_i, \hat{y}_j$):

$$\begin{aligned} \text{MemShares}_\sigma[q] &\leftarrow \text{Mult}(\text{MemShares}_\sigma[j], \text{MemShares}_\sigma^{\text{auth}}[j], c_i, \text{crs}_\ell) \\ \text{MemShares}_\sigma^{\text{auth}}[q] &\leftarrow \text{Mult}(\text{MemShares}_\sigma[j], \text{MemShares}_\sigma^{\text{auth}}[j], c_i^{\text{auth}}, \text{crs}_\ell) \end{aligned}$$

$$\text{MemShares}_\sigma[q] \leftarrow \text{DDLog}_{\text{pk}}((c_{\text{PRF},i}^{\text{fst}})^{\text{MemShares}_\sigma^{\text{auth}}[j]} \cdot (c_{\text{PRF},i}^{\text{snd}})^{\text{MemShares}_\sigma[j]})$$

$$\text{MemShares}_\sigma^{\text{auth}}[q] \leftarrow \text{DDLog}_{\text{pk}}((c_i^{\text{auth}})^{\text{fst}})^{\text{MemShares}_\sigma^{\text{auth}}[j]} \cdot (c_i^{\text{auth}})^{\text{snd}})^{\text{MemShares}_\sigma[j]}$$

- If op_ℓ is of the form (“Output”, $\hat{y}_i$)

$$y_0 \leftarrow \text{MemShares}_\sigma[i]$$

6. Output y_0

Fig. 7: Share conversion procedure for RMS programs with leading term.

Lemma 8 (Correctness of share conversion for RMS with leading multiplication, adapted from [CMPR23]). *Let $\lambda \in \mathbb{N}^*$ be a security parameter, let $\beta, B = \lambda^{\mathcal{O}(1)}$ be bounds, let Π be a B -bounded size- M RMS program on n inputs, and consider the deterministic algorithm ShareConvExtRMS of fig. 7 parameterised by Damgård-Jurik parameter ζ . For every $X \in [0, \beta)$ and all admissible inputs $x_1, \dots, x_n \in [0, B)$,*

$$\Pr \left[y_1 - y_0 = X \cdot \Pi(x_1, \dots, x_n) : (y_0, y_1) \stackrel{\$}{\leftarrow} \text{Sam}(1^\lambda, (x_i)_{i=1}^n) \right] \geq 1 - \frac{2M\beta B}{N^{\zeta-1}},$$

where the sampler Sam is defined as:

$\text{Sam}(1^\lambda, (x_i)_{i \in [n]}):$

- | | |
|---|---|
| <ol style="list-style-type: none"> 1. $((k, N), \text{pk}) \stackrel{\\$}{\leftarrow} \text{DJE.KeyGen}(1^\lambda)$ 2. $\text{sh}_0 \stackrel{\\$}{\leftarrow} [0, 2^\lambda B)$ 3. $\text{sh}_1 \leftarrow X + \text{sh}_{0,i}$ 4. $\text{sh}_0^{\text{auth}} \stackrel{\\$}{\leftarrow} [0, N \cdot 2^\lambda B)$ 5. $\text{sh}_1^{\text{auth}} \leftarrow k \cdot X + \text{sh}_{0,i}$ 6. $c_i \stackrel{\\$}{\leftarrow} \text{DJE.Enc}_{\text{pk}}(x_i)$ | <ol style="list-style-type: none"> 7. $c_i^{\text{auth}} \stackrel{\\$}{\leftarrow} \text{DJE.Enc}_{\text{pk}}(\text{sk} \cdot x_i)$ 8. $\text{crs} \stackrel{\\$}{\leftarrow} (\mathbb{Z}/N^\zeta\mathbb{Z})^M$ 9. For $\sigma \in \{0, 1\}$,
 $y_\sigma \leftarrow \text{ShareConvExtRMS}(\sigma, \text{pk}, \Pi, \text{sh}_\sigma, \text{sh}_\sigma^{\text{auth}}, (c_i)_{i \in [n]}, (c_i^{\text{auth}})_{i \in [n]}, \text{crs})$ 10. Output (y_0, y_1) |
|---|---|

Lemma 8 follows from [CMPR23] (without using the formalism of staged HSS).

5 Privately Constrained PRF for Bounded Dual Waring Rank

We are finally ready to present our construction of a privately constrained PRF for constraints with polynomially bounded degree and Waring rank. As the main intuition behind the constructions and its security were already provided in the introduction, we only provide the formal protocol description and formal theorem statement here.

PCPRF Bounded Dual Waring-rank constrained PRF

Requires:

- r and d are fixed bounds on the Waring rank and the degree of the dual constraints
- DJE is the cryptosystem of fig. 2
- ShareConv is the algorithm of fig. 6
- $\mathcal{C} \subseteq \{0, 1\}^n \rightarrow \{0, 1\}$ is a class of constraints whose dual have Waring rank at most r and degree at most d ; namely $\text{Encode}: \mathcal{C} \rightarrow \{0, 1\}^m$ is an encoding function, $\mathcal{C}^\perp := \{C_x: \{0, 1\}^m \rightarrow \{0, 1\}, \widehat{C} \mapsto C(x)\}$ is the dual class of $\mathcal{C}$, and Decompose is a deterministic function mapping each $C_x \in \mathcal{C}^\perp$ to one of its Waring-rank- r decompositions $(\alpha_i, d_i, f_i)_{i \in [r]}$.
- $\text{PRF}: \{0, 1\}^n \times \{0, 1\}^\lambda \rightarrow [0, 2^\mu)$ is a PRF which can be expressed RMS program of polynomial size M .
- ShareConvExtRMS is the algorithm of fig. 7.

$\text{CPRF.KeyGen}(1^\lambda):$

1. $(\text{sk} = (k, N), \text{pk} = (g, h, N)) \stackrel{\$}{\leftarrow} \text{DJE.KeyGen}(1^\lambda)$
2. $k_{\text{PRF}} = (k_{\text{PRF},1}, \dots, k_{\text{PRF},\lambda}) \stackrel{\$}{\leftarrow} \{0, 1\}^\lambda$

3. $c_{\text{cst}} \xleftarrow{\$} \text{DJE.Enc}_{\text{pk}}(1)$
 4. $c_{\text{cst}}^{\text{auth}} \xleftarrow{\$} \text{DJE.Enc}_{\text{pk}}(k)$
 5. For $i \in [m]$:
 - $\text{sh}_{0,i} \xleftarrow{\$} [0, 2^\lambda) // \langle \widehat{C}_i \rangle_0$ (λ -bit share over $\mathbb{Z}$)
 - $\text{sh}_{0,i}^{\text{auth}} \xleftarrow{\$} [0, N2^\lambda) // \langle k \cdot \widehat{C}_i \rangle_0$ ($(\lambda + \log N)$ -bit share over $\mathbb{Z}$)
 - $c_i \xleftarrow{\$} \text{DJE.Enc}_{\text{pk}}(\text{sh}_{0,i}) // \text{Enc}_{\text{pk}}(\langle \widehat{C}_i \rangle_0)$
 - $c_i^{\text{auth}} \xleftarrow{\$} \text{DJE.Enc}_{\text{pk}}(\text{sh}_{0,i}^{\text{auth}}) // \text{Enc}_{\text{pk}}(\langle k \cdot \widehat{C}_i \rangle_0)$
 - $c_i^{\text{ext-auth}} \xleftarrow{\$} \text{DJE.Enc}_{\text{pk}}(k \cdot \text{sh}_{0,i}) // \text{Enc}_{\text{pk}}(k \cdot \langle \widehat{C}_i \rangle_0)$
 6. For $i \in [\lambda]$:
 - $c_{\text{PRF},i} \xleftarrow{\$} \text{DJE.Enc}_{\text{pk}}(k_{\text{PRF},i})$
 - $c_{\text{PRF},i}^{\text{ext-auth}} \xleftarrow{\$} \text{DJE.Enc}_{\text{pk}}(k \cdot k_{\text{PRF},i})$
 7. $\text{crs} = (\text{crs}_1, \text{crs}_2) \xleftarrow{\$} (\mathbb{Z}/N^\zeta\mathbb{Z})^{4 \cdot \text{rd}} \times (\mathbb{Z}/N^\zeta\mathbb{Z})^M //$ A uniformly random string.
 8. Output $\text{msk} \leftarrow (\text{sk}, k_{\text{PRF}}, c_{\text{cst}}, c_{\text{cst}}^{\text{auth}},$
 $(\text{sh}_{0,i}, \text{sh}_{0,i}^{\text{auth}}, c_i, c_i^{\text{auth}}, c_i^{\text{ext-auth}})_{i \in [m]},$
 $(c_{\text{PRF},i}, c_{\text{PRF},i}^{\text{ext-auth}})_{i \in [\lambda]},$
 $\text{crs})$
 9. Output msk
- CPRF.Eval($1^\lambda, \text{msk}, x$) :**
1. Parse $\text{sk} \leftarrow (k, \text{pk})$
 2. $(\alpha_i, d_i, f_i)_{i \in [r]} \leftarrow \text{Decompose}(C_x)$
 3. Parse $\text{msk} = (\text{sk}, k_{\text{PRF}}, c_{\text{cst}}, c_{\text{cst}}^{\text{auth}},$
 $(\text{sh}_{0,i}, \text{sh}_{0,i}^{\text{auth}}, c_i, c_i^{\text{auth}}, c_i^{\text{ext-auth}})_{i \in [m]},$
 $(c_{\text{PRF},i}, c_{\text{PRF},i}^{\text{ext-auth}})_{i \in [\lambda]},$
 $\text{crs} = (\text{crs}_1, \text{crs}_2))$
 4. $(\overline{y}_0, \overline{y}_0^{\text{auth}}) \leftarrow \text{ShareConv}(0, k,$
 $(\alpha_i, f_i, d_i)_{i \in [r]},$
 $(\text{sh}_{0,i})_{i \in [m]},$
 $(\text{sh}_{0,i}^{\text{auth}})_{i \in [m]},$
 $(c_i)_{i \in [m]},$
 $(c_i^{\text{auth}})_{i \in [m]},$
 $(c_i^{\text{ext-auth}})_{i \in [m]},$
 $(c_{\text{cst}}, c_{\text{cst}}^{\text{auth}}),$
 $\text{crs}_1)$
 5. $y_0 \leftarrow \text{ShareConvExtRMS}(0, N, \text{PRF}, \overline{y}_0, \overline{y}_0^{\text{auth}}, (c_i)_{i \in [m]}, (c_i^{\text{ext-auth}})_{i \in [m]}, \text{crs}_2)$
 6. Output $y_0 \bmod 2^\mu$
- CPRF.Constrain($1^\lambda, \text{msk}, C$) :**
1. $\widehat{C} = (\widehat{C}_1, \dots, \widehat{C}_m) \leftarrow \text{Encode}(C)$
 2. Parse $\text{msk} = (\text{sk}, k_{\text{PRF}}, c_{\text{cst}}, c_{\text{cst}}^{\text{auth}},$
 $(\text{sh}_{0,i}, \text{sh}_{0,i}^{\text{auth}}, c_i, c_i^{\text{auth}}, c_i^{\text{ext-auth}})_{i \in [m]},$
 $(c_{\text{PRF},i}, c_{\text{PRF},i}^{\text{ext-auth}})_{i \in [\lambda]},$
 $\text{crs})$
 3. For $i \in [m]$,
 - $\text{sh}_{1,i} \leftarrow \widehat{C}_i + \text{sh}_{0,i}$
 - $\text{sh}_{1,i}^{\text{auth}} \leftarrow \text{sk} \cdot \widehat{C}_i + \text{sh}_{0,i}^{\text{auth}}$

```

4.  $\text{ck} \leftarrow (\text{pk}, c_{\text{cst}}, c_{\text{cst}}^{\text{auth}},$ 
    $(\text{sh}_{1,i}, \text{sh}_{1,i}^{\text{auth}}, c_i, c_i^{\text{auth}}, c_i^{\text{ext-auth}})_{i \in [m]},$ 
    $(c_{\text{PRF},i}, c_{\text{PRF},i}^{\text{ext-auth}})_{i \in [\lambda]},$ 
    $\text{crs})$ 
5. Output  $\text{ck}$ 

 $\text{CPRF.CEval}(1^\lambda, \text{ck}, x) :$ 

1. Parse  $\text{ck} = (\text{pk}, c_{\text{cst}}, c_{\text{cst}}^{\text{auth}},$ 
    $(\text{sh}_{1,i}, \text{sh}_{1,i}^{\text{auth}}, c_i, c_i^{\text{auth}}, c_i^{\text{ext-auth}})_{i \in [m]},$ 
    $(c_{\text{PRF},i}, c_{\text{PRF},i}^{\text{ext-auth}})_{i \in [\lambda]},$ 
    $\text{crs} = (\text{crs}_1, \text{crs}_2))$ 
2.  $(\bar{y}_1, \bar{y}_1^{\text{auth}}) \leftarrow \text{ShareConv}(1, \perp,$ 
    $(\alpha_i, f_i, d_i)_{i \in [r]},$ 
    $(\text{sh}_{1,i})_{i \in [m]},$ 
    $(\text{sh}_{1,i}^{\text{auth}})_{i \in [m]},$ 
    $(c_i)_{i \in [m]},$ 
    $(c_i^{\text{auth}})_{i \in [m]},$ 
    $(c_i^{\text{ext-auth}})_{i \in [m]},$ 
    $(c_{\text{cst}}, c_{\text{cst}}^{\text{auth}}),$ 
    $\text{crs}_1)$ 
3.  $y_1 \leftarrow \text{ShareConvExtRMS}(1, N, \text{PRF}, \bar{y}_1, \bar{y}_1^{\text{auth}}, (c_i)_{i \in [m]}, (c_i^{\text{ext-auth}})_{i \in [m]}, \text{crs}_2)$ 
4. Output  $y_1 \bmod 2^\mu$ 

```

Fig. 8: Privately Constrained PRF for any class of constraints whose dual have (polynomially) bounded Waring rank and degree.

Main Theorem 1 (Privately Constrained PRF for Bounded Dual Waring Rank from DCR). *Let $\mathcal{C}$ be any class of constraints whose duals all have Waring rank and algebraic degree upper bounded by some polynomials in the security parameter. The construction of fig. 8 is a single-key, selectively secure privately constrained PRF for $\mathcal{C}$.*

We refer to appendix B.2 for the proof of main theorem 1.

Corollary 9 (Privately Punctured PRF from DCR). *Assuming DCR, there exists a single-key, selectively secure, privately punctured PRF.*

Proof. As already discussed in the technical overview, the puncturing constraint can be expressed with a polynomial of bounded degree and Waring rank. More formally, the class of puncturing constraints is¹⁶

$$\mathcal{C} := \{C_{x^*} : \{0, 1\}^n \rightarrow \{0, 1\}, x \mapsto x = x^* \mid x^* \in \{0, 1\}^n\}$$

and it coincides with its own dual class

$$\mathcal{C}^\perp := \{C_x : \{0, 1\}^n \rightarrow \{0, 1\}, x^* \mapsto x = x^* \mid x \in \{0, 1\}^n\}$$

We show that the puncturing constraint C_x can be expressed as a polynomial of degree n in the bits of x^* : We can define a function A_x that counts the number of common bits

¹⁶ Remember we want $C(x) = 0$ for authorised inputs, and $C(x) = 1$ for constrained inputs.

between x, x^* and rewrite the puncturing constraint as a the product of these terms, and then normalise it by the max value of this polynomial to ensure that $C_x(x^*) = 1$. More formally:

$$\begin{aligned}
C_x(x^*) &= \left[\sum_{i=1}^n (x_i = x_i^*) = n \right] \\
&= \left[\prod_{i=0}^{n-1} (A_x(x^*) - i) \neq 0 \right] \quad \text{where } A_x: \{0, 1\}^n \rightarrow \mathbb{Z} \\
&\quad (X_i)_{i \in [n]} \mapsto \sum_{i=1}^n (x_i = X_i) \\
&= \left[\sum_{i=0}^n \alpha_i \cdot A_x(x^*)^i \right] / (n!) \\
&= \sum_{i=0}^n \frac{\alpha_i}{n!} \cdot A_x(x^*)^i
\end{aligned}$$

where the $(\alpha_i)_{i=0}^n$ are the coefficients¹⁷ of the degree- n polynomial $\prod_{i=0}^{n-1} (X - i)$. In this form, the dual of puncturing clearly has Waring rank at most n . $\square$

6 Waring Rank

In this section, we provide some background on Waring rank and prove some results that are useful for our construction. We will relax the homogeneity condition and define Waring rank for polynomials of degree *at most* d .

Definition 10 (Waring Rank and its variants). *Let $f \in \mathbb{Q}[X_1, \dots, X_N]$ be a polynomial of degree $d \in \mathbb{N}$. In the following, let $f_1, \dots, f_r, g_1, \dots, g_r \in \mathbb{Q}[X_1, \dots, X_N]$ denote affine functions¹⁸ and $\alpha_1, \dots, \alpha_r \in \mathbb{Q}$.*

- *The Waring rank of f is defined to be the smallest integer $r \in \mathbb{N}$ such that there exists affine functions $f_1, \dots, f_r$ and coefficients $\alpha_1, \dots, \alpha_r$ satisfying*

$$f(X_1, \dots, X_N) = \sum_{i=1}^r \alpha_i (f_i(X_1, \dots, X_N))^d.$$

We will refer to the set $\{(f_1, \alpha_1), \dots, (f_r, \alpha_r)\}$ as a Waring decomposition of f of rank r .

- *The Mixed-degree Waring rank of f is defined to be the smallest integer $r \in \mathbb{N}$ such that there exists affine functions $f_1, \dots, f_r$ and coefficients $\alpha_1, \dots, \alpha_r$ satisfying*

$$f(X_1, \dots, X_N) = \sum_{i=1}^r \alpha_i (f_i(X_1, \dots, X_N))^{d_i}, \quad \text{where each } 0 \leq d_i \leq d.$$

We will refer to the set $\{(f_1, d_1, \alpha_1), \dots, (f_r, d_r, \alpha_r)\}$ as a Mixed Waring decomposition of f of rank r .

¹⁷ Note that these coefficients can be computed in polynomial time.

¹⁸ We allow affine (instead of linear) functions here because we are concerned with representing inhomogeneous polynomials 7.

- The Split Waring Rank of f is defined to be the smallest integer $r \in \mathbb{N}$ such that there exists affine functions $f_1, \dots, f_r$ and $g_1, \dots, g_r$ and coefficients $\alpha_1, \dots, \alpha_r$ satisfying

$$f(X_1, \dots, X_N) = \sum_{i=1}^r \alpha_i (f_i(X_1, \dots, X_{N/2}))^{d_i} \cdot (g_i(X_{N/2+1}, \dots, X_N))^{e_i},$$

where for each $i \in [r]$, $d_i, e_i \geq 0$ and $(d_i + e_i) \leq d$. We will refer to the set $\{(f_1, g_1, d_1, e_1, \alpha_1), \dots, (f_r, g_r, d_r, e_r, \alpha_r)\}$ as a Split Waring decomposition of f of rank r .

An example. We give two simple examples of Waring decomposition for concreteness.

$$X_1 X_2 = \frac{1}{4}(X_1 + X_2)^2 - \frac{1}{4}(X_1 - X_2)^2$$

$$\begin{aligned} X_1 X_2 X_3 &= \frac{1}{24}(X_1 + X_2 + X_3)^3 - \frac{1}{24}(X_1 + X_2 - X_3)^3 \\ &\quad - \frac{1}{24}(X_1 - X_2 + X_3)^3 - \frac{1}{24}(-X_1 + X_2 + X_3)^3 \end{aligned}$$

At first sight, it is unclear whether the Waring rank of every degree $\leq d$ polynomial is finite. However, extending the above examples, it is not difficult to show that every monomial of degree $\leq d$ has a finite Waring rank, and this shows that every degree $\leq d$ polynomial has a finite Waring rank (using the fact that monomials of degree $\leq d$ span the space of polynomials of degree $\leq d$). For a proof of this see e.g., [Tei14].

Relation Between Notions. We will first show that for a polynomial f of degree d , the Waring rank, Mixed Waring rank, and Split Waring rank are equivalent up to $\text{poly}(d)$ factors, i.e. if f has a low-rank Waring, Mixed Waring, Split Waring decomposition, then it has a low-rank Waring, Mixed Waring, and Split Waring decomposition (see Lemma 12). To show this, we will use the following lemma.

Lemma 11. *Let $\ell_1, \ell_2 \in \mathbb{Q}[X_1, \dots, X_N]$ be affine functions. For any $d_1, d_2 \in \mathbb{N}$, the Waring rank of $\ell_1^{d_1} \cdot \ell_2^{d_2}$ is $\leq (d_1 + d_2 + 1)$.*

Proof of Lemma 11. Consider the following univariate polynomial:

$$A(t) := (\ell_1(X_1, \dots, X_N) + t \cdot \ell_2(X_1, \dots, X_N))^{d_1+d_2}$$

Note that coefficient of t^{d_2} in the univariate polynomial $A(t)$ is exactly $\binom{d_1+d_2}{d_1} \ell_1^{d_1} \ell_2^{d_2}$. Using interpolation on $A(t)$, we can express each coefficient of $A(t)$ as a linear combination of evaluations of $A(t)$. More precisely, there exists coefficients $c_0, \dots, c_{d_1+d_2}$ such that the following holds:

$$\text{coefficient of } t^{d_2} \text{ in } A(t) = \sum_{j=0}^{d_1+d_2} c_j A(j) = \sum_{j=0}^{d_1+d_2} c_j (\ell_1 + j \ell_2)^{d_1+d_2}.$$

So we have expressed the coefficient of t^{d_2} as a sum of the $(d_1 + d_2)^{\text{th}}$ power of $(d_1 + d_2 + 1)$ affine functions. As noted before, the coefficient of t^{d_2} in $A(t)$ is a non-zero scalar multiple of $(\ell_1(X_1, \dots, X_N))^{d_1} \cdot (\ell_2(X_1, \dots, X_N))^{d_2}$. This completes the proof of Lemma 11. $\square$

Now we are ready to show that the ranks are equivalent up to $\text{poly}(d)$ factors.

Lemma 12. *Let $f \in \mathbb{Q}[X_1, \dots, X_N]$ be a polynomial of degree $\leq d$. If any one of the Waring rank/Mixed Waring rank/Split Waring rank of f is $\leq r$, then the remaining two are at most $\mathcal{O}(rd^2)$. In simple words, all three ranks are equivalent up to a multiplicative factor of $\text{poly}(d)$.*

Proof of Lemma 12. We start by observing that if $f(X_1, \dots, X_N)$ has Waring rank r , then it has Mixed Waring rank $\leq r$ (every Waring decomposition is also a Mixed Waring decomposition). We now show that if f has Waring rank r , then it has Split Waring rank $\leq r(d+1)$. Suppose f has a Waring decomposition $\{(f_1, \alpha_1), \dots, (f_r, \alpha_r)\}$ of rank r , then we “split” each affine function f_i as a sum of two affine functions $f_{i,1}$ and $f_{i,2}$ on disjoint sets of variables, i.e.

$$f_i(X_1, \dots, X_N) = f_{i,1}(X_1, \dots, X_{N/2}) + f_{i,2}(X_{N/2+1}, \dots, X_N).$$

Using Binomial Theorem on $f_i^d = (f_{i,1} + f_{i,2})^d$ for each $i \in [r]$, we have,

$$(f_i(X_1, \dots, X_N))^d = \sum_{j=0}^d \binom{d}{j} (f_{i,1}(X_1, \dots, X_{N/2}))^j \cdot (f_{i,2}(X_{N/2+1}, \dots, X_N))^{d-j}$$

This gives us a Split Waring decomposition of f of rank $\leq r(d+1)$.

Suppose $f(X_1, \dots, X_N)$ has Mixed Waring rank r . We will now argue that $f(X_1, \dots, X_N)$ has Waring rank $\leq r(d+1)$. Suppose f has a Mixed Waring decomposition $\{(f_1, d_1, \alpha_1), \dots, (f_r, d_r, \alpha_r)\}$ of rank r . Define the polynomial $A(X_1, \dots, X_N, Z)$ as follows:

$$A(X_1, \dots, X_N, Z) = \sum_{i=1}^r \alpha_i (f_i(X_1, \dots, X_N))^{d_i} \cdot Z^{d-d_i}, \quad \text{where each } 0 \leq d_i \leq d$$

Applying Lemma 11 on $f_i^{d_i} \cdot Z^{d-d_i}$ for each $i \in [r]$, we get that $A(X_1, \dots, X_N, Z)$ has a Waring decomposition which has $r(d+1)$ affine functions in $\mathbb{Q}[X_1, \dots, X_N, Z]$. Observe that $A(X_1, \dots, X_N, 1) = f(X_1, \dots, X_N)$. Setting $Z = 1$ shows that $f(X_1, \dots, X_N)$ has Waring rank $\leq r(d+1)$.

Suppose $f(X_1, \dots, X_N)$ has Split Waring rank r . We will now argue that $f(X_1, \dots, X_N)$ has Waring rank $\leq r(d+1)^2$. Suppose f has a Split Waring decomposition $\{(f_1, g_1, d_1, e_1, \alpha_1), \dots, (f_r, g_r, d_r, e_r, \alpha_r)\}$ of rank r . Applying Lemma 11 on $f_i^{d_i} g_i^{e_i}$ for each $i \in [r]$, we get that the Mixed Waring rank of $f(X_1, \dots, X_N)$ is $\leq \sum_{i=1}^r (d_i + e_i + 1)$. Now using the discussion from the previous paragraph, we conclude that the Waring rank of $f(X_1, \dots, X_N)$ is $\leq (d+1) \sum_{i=1}^r (d_i + e_i + 1)$, which is $\leq r(d+1)^2$.

To finish the proof, we need to argue that the Mixed Waring rank and the Split Waring rank are equivalent. If f has Mixed Waring rank $\leq r$, then it has Waring rank $\leq r(d+1)$, which implies it has Split Waring rank $\leq r(d+1)^2$. Similarly, if f has Split Waring rank $\leq r$, then it has Waring rank $\leq r(d+1)^2$, which implies it has Mixed Waring rank $\leq r(d+1)^2$. This finishes the proof of Lemma 12. $\square$

Lemma 13 shows that if two polynomials have a small Waring rank, then their product also has a small Waring rank.

Lemma 13. *Let $f, g \in \mathbb{Q}[X_1, \dots, X_N]$ be polynomials with Waring ranks (resp. mixed-degree Waring ranks) r_f and r_g . Then the polynomial fg has Waring rank (resp. mixed-degree Waring rank) $\leq r_f r_g (\deg(f) + \deg(g) + 1)$.*

Proof. Suppose f and g have Waring decompositions $\{(f_1, \alpha_1), \dots, (f_{r_f}, \alpha_{r_f})\}$ and $\{(g_1, \alpha_1), \dots, (g_{r_g}, \alpha_{r_g})\}$ of ranks r_f and r_g respectively. Then,

$$f(X_1, \dots, X_N) \cdot g(X_1, \dots, X_N) = \sum_{\substack{1 \leq i \leq r_f \\ 1 \leq j \leq r_g}} \alpha_i \beta_j (f_i(X_1, \dots, X_N))^{\deg(f)} \cdot (g_j(X_1, \dots, X_N))^{\deg(g)}.$$

Now applying Lemma 11 on $f_i^{\deg(f)} g_j^{\deg(g)}$ for each $1 \leq i \leq r_f$ and $1 \leq j \leq r_g$, we get that the Waring rank of $f(X_1, \dots, X_N) \cdot g(X_1, \dots, X_N)$ is $\leq r_f r_g (\deg(f) + \deg(g) + 1)$.

An analogous argument shows that if f and g have Mixed Waring rank r_f and r_g , then $f(X_1, \dots, X_N) \cdot g(X_1, \dots, X_N)$ have Mixed Waring rank $\leq r_f r_g (\deg(f) + \deg(g) + 1)$. This finishes the proof of Lemma 13. $\square$

A crucial part of our construction of a privately constrained PRF is the duality between seeing the constraint as a function (applied to the public input) and casting it as the secret input to a public function (a universal circuit with hardcoded input). In theorem 14 we unify these perspectives by showing that the class of functions with polynomial degree and Waring rank (the functions being encoded as Waring coefficient vector) is closed under duality.

Theorem 14 (Duality). *For every $d, r \in \mathbb{N}$, there exists an encoding $\rho : \mathbb{Q}[X_1, \dots, X_N] \rightarrow \mathbb{Q}^M$ of all polynomials $f \in \mathbb{Q}[X_1, \dots, X_N]$ with $\deg(f) \leq d$ and Waring rank $\leq r$, where $M = (r + 1)N$.*

Additionally, for every $(x_1, \dots, x_N) \in \mathbb{Q}^N$, there exists a polynomial $g_{x_1, \dots, x_N} \in \mathbb{Q}[Y_1, \dots, Y_M]$ of degree $\leq d$ such that $g_{x_1, \dots, x_N}(\rho(f)) = f(x_1, \dots, x_N)$, and Waring rank of $g_{x_1, \dots, x_N}$ is $\leq r$.

Proof. We first describe an encoding ρ . Suppose f is a degree $\leq d$ polynomial with Waring rank $\leq r$, with the following Waring decomposition¹⁹:

$$f(X_1, \dots, X_N) = \sum_{i=1}^r \alpha_i (f_i(X_1, \dots, X_N))^d, \quad (3)$$

where for every $i \in [r]$, the affine function $f_i \in \mathbb{Q}[X_1, \dots, X_N]$ is

$$f_i(X_1, \dots, X_N) = c_{i,0} + c_{i,1}X_1 + \dots + c_{i,N}X_N. \quad (4)$$

The map ρ maps $f(X_1, \dots, X_N)$ to $\mathbb{Q}^M$ as follows:

$$\rho(f(X_1, \dots, X_N)) = (c_{1,0}, \dots, c_{1,N}, \dots, c_{r,0}, \dots, c_{r,N}).$$

In simple words, ρ simply maps f to a M -dimensional vector which has all the coefficients of every affine function f_i in a (fixed) Waring decomposition of f of rank r .

Next we describe the polynomial $g_{x_1, \dots, x_N}$. Fix an arbitrary $(x_1, \dots, x_N) \in \mathbb{Q}^N$. For each $i \in [r]$, define a new polynomial $\tilde{f}_i \in \mathbb{Q}[X_1, \dots, X_N][Y_1, \dots, Y_M]$ (i.e. $\tilde{f}_i$ is a polynomial in Y -variables with coefficients from the ring $\mathbb{Q}[X_1, \dots, X_N]$) as follows: We replace $c_{i,j}$ with the variable $Y_{(i-1)(N+1)+(j+1)}$. In other words, for each $i \in [r]$,

$$\tilde{f}_i(X_1, \dots, X_N, Y_1, \dots, Y_M) = Y_{(i-1)N+1} + Y_{(i-1)N+2}X_1 + \dots + Y_{(i-1)N+(N+1)}X_N.$$

Similarly, using Equation (3), let $\tilde{f} := \sum_{i=1}^r \alpha_i \tilde{f}_i$. Observe that $\tilde{f}_i$'s have degree 1 in the Y variables, and $\tilde{f}$ is a polynomial of degree $\leq d$ in the Y variables.

¹⁹ If there are multiple Waring decompositions of f , we choose one decomposition arbitrarily.

Define the polynomial $g_{x_1, \dots, x_N} \in \mathbb{Q}[Y_1, \dots, Y_M]$ as follows:

$$g_{x_1, \dots, x_N}(Y_1, \dots, Y_M) := \tilde{f}(x_1, \dots, x_N, Y_1, \dots, Y_M).$$

By definition, $\tilde{f}(x_1, \dots, x_N, \rho(f)) = f(x_1, \dots, x_N)$, which implies that $g_{x_1, \dots, x_N}(\rho(f)) = f(x_1, \dots, x_N)$. Since the degree of $\tilde{f}$ in Y -variables is $\leq d$, this implies that the degree of $g_{x_1, \dots, x_N}$ is $\leq d$. Now it remains to show that the Waring rank of $g_{x_1, \dots, x_N}$ is $\leq r$. Using Equation (3), it is easy to verify:

$$g_{x_1, \dots, x_N}(Y_1, \dots, Y_M) = \sum_{i=1}^r \alpha_i \tilde{f}_i(x_1, \dots, x_N, Y_1, \dots, Y_M).$$

Note that for each $i \in [r]$, the polynomial $\tilde{f}_i(x_1, \dots, x_N, Y_1, \dots, Y_M) \in \mathbb{Q}[Y_1, \dots, Y_M]$ is an affine function. In other words, we have given a Waring decomposition of $\tilde{f}$ of rank r . This finishes the proof of Theorem 14. $\square$

Acknowledgments

We thank Geoffroy Couteau for insightful discussions.

This research was supported by the European Research Council (ERC) under the European Union’s Horizon 2020 research and innovation programme under grant agreement number 101124977 (DECRYPSTYS), and the Danish Independent Research Council under Grant-IDs DFF-3103-00077B (CryptoDigi) and DFF-0165-00107B (C3PO). Amik Raj Behera is supported by Srikanth Srinivasan’s start-up grant from the University of Copenhagen.

References

- ADOS22. Damiano Abram, Ivan Damgård, Claudio Orlandi, and Peter Scholl. An algebraic framework for silent preprocessing with trustless setup and active security. In Yevgeniy Dodis and Thomas Shrimpton, editors, *CRYPTO 2022, Part IV*, volume 13510 of *LNCS*, pages 421–452. Springer, Cham, August 2022.
- AMN⁺18. Nuttapong Attrapadung, Takahiro Matsuda, Ryo Nishimaki, Shota Yamada, and Takashi Yamakawa. Constrained PRFs for NC¹ in traditional groups. In Hovav Shacham and Alexandra Boldyreva, editors, *CRYPTO 2018, Part II*, volume 10992 of *LNCS*, pages 543–574. Springer, Cham, August 2018.
- BCG⁺19. Elette Boyle, Geoffroy Couteau, Niv Gilboa, Yuval Ishai, Lisa Kohl, and Peter Scholl. Efficient pseudorandom correlation generators: Silent OT extension and more. In Alexandra Boldyreva and Daniele Micciancio, editors, *CRYPTO 2019, Part III*, volume 11694 of *LNCS*, pages 489–518. Springer, Cham, August 2019.
- BCM⁺24. Dung Bui, Geoffroy Couteau, Pierre Meyer, Alain Passelègue, and Mahshid Riahinia. Fast public-key silent OT and more from constrained Naor-Reingold. In Marc Joye and Gregor Leander, editors, *EUROCRYPT 2024, Part VI*, volume 14656 of *LNCS*, pages 88–118. Springer, Cham, May 2024.
- BGI14. Elette Boyle, Shafi Goldwasser, and Ioana Ivan. Functional signatures and pseudorandom functions. In Hugo Krawczyk, editor, *PKC 2014*, volume 8383 of *LNCS*, pages 501–519. Springer, Berlin, Heidelberg, March 2014.
- BGI16. Elette Boyle, Niv Gilboa, and Yuval Ishai. Breaking the circuit size barrier for secure computation under DDH. In Matthew Robshaw and Jonathan Katz, editors, *CRYPTO 2016, Part I*, volume 9814 of *LNCS*, pages 509–539. Springer, Berlin, Heidelberg, August 2016.
- BGIK22. Elette Boyle, Niv Gilboa, Yuval Ishai, and Victor I. Kolobov. Programmable distributed point functions. In Yevgeniy Dodis and Thomas Shrimpton, editors, *CRYPTO 2022, Part IV*, volume 13510 of *LNCS*, pages 121–151. Springer, Cham, August 2022.
- BLW17. Dan Boneh, Kevin Lewi, and David J. Wu. Constraining pseudorandom functions privately. In Serge Fehr, editor, *PKC 2017, Part II*, volume 10175 of *LNCS*, pages 494–524. Springer, Berlin, Heidelberg, March 2017.

- BTWV17. Zvika Brakerski, Rotem Tsabary, Vinod Vaikuntanathan, and Hoeteck Wee. Private constrained PRFs (and more) from LWE. In Yael Kalai and Leonid Reyzin, editors, *TCC 2017, Part I*, volume 10677 of *LNCS*, pages 264–302. Springer, Cham, November 2017.
- BW13. Dan Boneh and Brent Waters. Constrained pseudorandom functions and their applications. In Kazue Sako and Palash Sarkar, editors, *ASIACRYPT 2013, Part II*, volume 8270 of *LNCS*, pages 280–300. Springer, Berlin, Heidelberg, December 2013.
- CC17. Ran Canetti and Yilei Chen. Constraint-hiding constrained PRFs for NC^1 from LWE. In Jean-Sébastien Coron and Jesper Buus Nielsen, editors, *EUROCRYPT 2017, Part I*, volume 10210 of *LNCS*, pages 446–476. Springer, Cham, April / May 2017.
- CHI⁺18. Luca Chiantini, Jonathan D. Hauenstein, Christian Ikenmeyer, Joseph M. Landsberg, and Giorgio Ottaviani. Polynomials and the exponent of matrix multiplication. *Bulletin of the London Mathematical Society*, 50(3):369–389, 2018.
- CMPR23. Geoffroy Couteau, Pierre Meyer, Alain Passelègue, and Mahshid Riahinia. Constrained pseudorandom functions from homomorphic secret sharing. In Carmit Hazay and Martijn Stam, editors, *EUROCRYPT 2023, Part III*, volume 14006 of *LNCS*, pages 194–224. Springer, Cham, April 2023.
- DGI⁺24. Pranjal Dutta, Fulvio Gesmundo, Christian Ikenmeyer, Gorav Jindal, and Vladimir Lysikov. Fixed-parameter debordering of waring rank. In Olaf Beyersdorff, Mamadou Moustapha Kanté, Orna Kupferman, and Daniel Lokshtanov, editors, *41st International Symposium on Theoretical Aspects of Computer Science, STACS 2024, March 12-14, 2024, Clermont-Ferrand, France*, volume 289 of *LIPIcs*, pages 30:1–30:15. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2024.
- DJ01. Ivan Damgård and Mats Jurik. A generalisation, a simplification and some applications of Paillier’s probabilistic public-key system. In Kwangjo Kim, editor, *PKC 2001*, volume 1992 of *LNCS*, pages 119–136. Springer, Berlin, Heidelberg, February 2001.
- DKN⁺20. Alex Davidson, Shuichi Katsumata, Ryo Nishimaki, Shota Yamada, and Takashi Yamakawa. Adaptively secure constrained pseudorandom functions in the standard model. In Daniele Micciancio and Thomas Ristenpart, editors, *CRYPTO 2020, Part I*, volume 12170 of *LNCS*, pages 559–589. Springer, Cham, August 2020.
- EGdOW18. Klim Efremenko, Ankit Garg, Rafael Mendes de Oliveira, and Avi Wigderson. Barriers for rank methods in arithmetic complexity. In Anna R. Karlin, editor, *9th Innovations in Theoretical Computer Science Conference, ITCS 2018, January 11-14, 2018, Cambridge, MA, USA*, volume 94 of *LIPIcs*, pages 1:1–1:19. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2018.
- GGM84. Oded Goldreich, Shafi Goldwasser, and Silvio Micali. How to construct random functions (extended abstract). In *25th FOCS*, pages 464–479. IEEE Computer Society Press, October 1984.
- HKKW19. Dennis Hofheinz, Akshay Kamath, Venkata Koppula, and Brent Waters. Adaptively secure constrained pseudorandom functions. In Ian Goldberg and Tyler Moore, editors, *FC 2019*, volume 11598 of *LNCS*, pages 357–376. Springer, Cham, February 2019.
- IK99. Anthony Iarrobino and Vassil Kanev. *Power Sums, Gorenstein Algebras, and Determinantal Loci*. Cambridge Studies in Advanced Mathematics. Springer Berlin, Heidelberg, 1999.
- ILL24. Yuval Ishai, Hanjun Li, and Huijia Lin. Succinct partial garbling from groups and applications. Cryptology ePrint Archive, Paper 2024/2073, to appear at Crypto 2025, 2024.
- KPTZ13. Aggelos Kiayias, Stavros Papadopoulos, Nikos Triandopoulos, and Thomas Zacharias. Delegatable pseudorandom functions and applications. In Ahmad-Reza Sadeghi, Virgil D. Gligor, and Moti Yung, editors, *ACM CCS 2013*, pages 669–684. ACM Press, November 2013.
- Lan17. J. M. Landsberg. *Geometry and Complexity Theory*. Cambridge Studies in Advanced Mathematics. Cambridge University Press, 2017.
- MORS24. Pierre Meyer, Claudio Orlandi, Lawrence Roy, and Peter Scholl. Rate-1 arithmetic garbling from homomorphic secret sharing. In Elette Boyle and Mohammad Mahmoody, editors, *TCC 2024, Part IV*, volume 15367 of *LNCS*, pages 71–97. Springer, Cham, December 2024.
- MORS25. Pierre Meyer, Claudio Orlandi, Lawrence Roy, and Peter Scholl. Silent circuit relinearisation: Sublinear-size (boolean and arithmetic) garbled circuits from DCR. Cryptology ePrint Archive, Report 2025/245, 2025.
- OSY21. Claudio Orlandi, Peter Scholl, and Sophia Yakubov. The rise of paillier: Homomorphic secret sharing and public-key silent OT. In Anne Canteaut and François-Xavier Standaert, editors, *EUROCRYPT 2021, Part I*, volume 12696 of *LNCS*, pages 678–708. Springer, Cham, October 2021.
- Pai99. Pascal Paillier. Public-key cryptosystems based on composite degree residuosity classes. In Jacques Stern, editor, *EUROCRYPT’99*, volume 1592 of *LNCS*, pages 223–238. Springer, Berlin, Heidelberg, May 1999.
- Pra19. Kevin Pratt. Waring rank, parameterized and exact algorithms. In David Zuckerman, editor, *60th IEEE Annual Symposium on Foundations of Computer Science, FOCS 2019, Baltimore, Maryland, USA, November 9-12, 2019*, pages 806–823. IEEE Computer Society, 2019.

- PS18. Chris Peikert and Sina Shiehian. Privately constraining and programming PRFs, the LWE way. In Michel Abdalla and Ricardo Dahab, editors, *PKC 2018, Part II*, volume 10770 of *LNCS*, pages 675–701. Springer, Cham, March 2018.
- RS21. Lawrence Roy and Jaspal Singh. Large message homomorphic secret sharing from DCR and applications. In Tal Malkin and Chris Peikert, editors, *CRYPTO 2021, Part III*, volume 12827 of *LNCS*, pages 687–717, Virtual Event, August 2021. Springer, Cham.
- Ser24. Sacha Servan-Schreiber. Constrained pseudorandom functions for inner-product predicates from weaker assumptions. Cryptology ePrint Archive, Paper 2024/058, 2024.
- SW14. Amit Sahai and Brent Waters. How to use indistinguishability obfuscation: deniable encryption, and more. In David B. Shmoys, editor, *46th ACM STOC*, pages 475–484. ACM Press, May / June 2014.
- Tei14. Zach Teitler. Ranks of polynomials. <https://www.theoremoftheday.org/Docs/Teitler2014.pdf>, 2014.

Supplementary Material

A More Preliminaries

A.1 Damgård-Jurik-ElGamal encryption scheme

Definition 15 (Decision Composite Residuosity Assumption (DCR), [Pai99]). Let RSA.Gen be a polynomial-time algorithm which, on input a security parameter λ , outputs (N, p, q) where p and q are λ -bit primes and $N = pq$. Let λ be a security parameter. We say that the Decision Composite Residuosity (DCR) problem is hard relative to modulus-sampling algorithm RSA.Gen if

$$\left\{ (N, x) : \begin{array}{l} (N, p, q) \xleftarrow{\$} \text{RSA.Gen}(1^\lambda) \\ x \xleftarrow{\$} (\mathbb{Z}/N^2\mathbb{Z})^\times \end{array} \right\} \stackrel{c}{\approx} \left\{ (N, x^N \bmod N^2) : \begin{array}{l} (N, p, q) \xleftarrow{\$} \text{RSA.Gen}(1^\lambda) \\ x \xleftarrow{\$} (\mathbb{Z}/N^2\mathbb{Z})^\times \end{array} \right\}.$$

Theorem 16. Assuming DCR, Damgård–Jurik–ElGamal encryption fig. 9 is a public key encryption scheme satisfying correctness and KDM security for affine functions of the key. Specifically, the following properties hold:

Correctness: $\text{DJE.Dec}_{\text{sk}}(\text{DJE.Enc}_{\text{pk}}(x)) = x$, for any (sk, pk) in the support of DJE.KeyGen , and any $x \in \mathbb{Z}/N^\zeta\mathbb{Z}$.

KDM Security: For all p.p.t. adversaries Adv , the oracles $\mathcal{O}_{\text{sk}, \text{pk}, R}^{\text{KDM}}$ and $\mathcal{O}_{\text{sk}, \text{pk}, \$}^{\text{KDM}}$ are indistinguishable in the following experiment

$$\begin{array}{l} (\text{sk}, \text{pk}) \xleftarrow{\$} \text{DJE.KeyGen}(1^\lambda) \\ \text{output } \text{Adv}^{\mathcal{O}_{\text{sk}, \text{pk}, R/\$}^{\text{KDM}}}(\text{pk}) \end{array}$$

where these oracles are defined as

$\begin{array}{l} \mathcal{O}_{\text{sk}, \text{pk}, R}^{\text{KDM}}(x, y): \\ (k, N) \leftarrow \text{sk} \\ z \leftarrow x \cdot k + y \\ \text{return } \text{DJE.Enc}_{\text{pk}}(z) \end{array}$	$\begin{array}{l} \mathcal{O}_{\text{sk}, \text{pk}, \$}^{\text{KDM}}(x, y): \\ (k, N) \leftarrow \text{sk} \\ z \xleftarrow{\$} \mathbb{Z}/N^\zeta\mathbb{Z} \\ \text{return } \text{DJE.Enc}_{\text{pk}}(z) \end{array}$
---	--

Proof. This is very similar to many previous encryption schemes used in HSS and proven KDM secure in that context. See, e.g., the Damgård–Jurik instantiation of the NIDLS framework [ADOS22]), though details differ, as we do not require p and q to be safe primes. For completeness, we present a full proof in appendix A.2. $\square$

DJE Damgård-Jurik-ElGamal Cryptosystem

Requires:

- $\zeta \geq 1$ is a parameter defining the plaintext size.
- Group isomorphism $\text{exp}: (\mathbb{Z}/N^\zeta\mathbb{Z})^+ \rightarrow 1 + N(\mathbb{Z}/N^{\zeta+1}\mathbb{Z})$ and its inverse $\text{log}: 1 + N(\mathbb{Z}/N^{\zeta+1}\mathbb{Z}) \rightarrow (\mathbb{Z}/N^\zeta\mathbb{Z})^+$, as defined as in [RS21]:

$$\text{exp}(x) = \sum_{k=0}^{\zeta} \frac{(Nx)^k}{k!} \quad \text{and} \quad \text{log}(1 + Nx) = \sum_{k=1}^{\zeta} \frac{(-N)^{k-1} x^k}{k}$$

DJE.KeyGen(1^λ): 1. Sample $N \xleftarrow{\\$} \text{RSA.Gen}(1^\lambda)$ 2. Sample $k \xleftarrow{\\$} [0, N)$ 3. Sample $g \xleftarrow{\\$} (\mathbb{Z}/N^{\zeta+1}\mathbb{Z})^\times$ 4. Compute $h \leftarrow g^{-k}$ 5. Output $(\text{sk} = (k, N), \text{pk} = (g, h, N))$ DJE.Dec_{sk}($c = (c_0, c_1)$): 1. Parse $\text{sk} = (k, N)$ 2. Assert $c_0^k \cdot c_1 \equiv 1 \pmod{N}$ 3. Output $x \leftarrow \log(c_0^k \cdot c_1)$	DJE.Enc_{pk}(x): 1. Parse $\text{pk} = (g, h, N)$ 2. Sample $r \xleftarrow{\\$} [0, N)$ 3. Compute $c_0 \leftarrow g^r$ 4. Compute $c_1 \leftarrow h^r \cdot \exp(x)$ 5. Output $c = (c_0, c_1)$
--	---

Fig. 9: The Damgård-Jurik-ElGamal cryptosystem.

A.2 Damgård-Jurik-ElGamal Cryptosystem Proofs

A.2.1 Correctness of Damgård-Jurik-ElGamal Correctness is straightforward from the definitions:

$$\begin{aligned}
\text{DJE.Dec}_{\text{sk}}(\text{DJE.Enc}_{\text{pk}}(x)) &= \text{DJE.Dec}_{\text{sk}}(g^r, (g^{-k})^r \cdot \exp(x)) \\
&= \log(g^{kr} \cdot (g^{-k})^r \cdot \exp(x)) \\
&= \log(\exp(x)) = x
\end{aligned}$$

A.2.2 DCR over $(\mathbb{Z}/N^{\zeta+1}\mathbb{Z})^\times$ Before proving security, we will need a lemma to extend the DCR assumption to the group $(\mathbb{Z}/N^{\zeta+1}\mathbb{Z})^\times$ that we are actually working over.

Lemma 17. *Assuming DCR, for any plaintext size parameter $\zeta \geq 1$ (which in general can be any polynomially-bounded PPT function of N), we have*

$$\left\{ (N, x): \begin{array}{l} (N, p, q) \xleftarrow{\$} \text{RSA.Gen}(1^\lambda) \\ x \xleftarrow{\$} (\mathbb{Z}/N^{\zeta+1}\mathbb{Z})^\times \end{array} \right\} \stackrel{c}{\approx} \left\{ (N, x^{N^\zeta} \bmod N^{\zeta+1}): \begin{array}{l} (N, p, q) \xleftarrow{\$} \text{RSA.Gen}(1^\lambda) \\ x \xleftarrow{\$} (\mathbb{Z}/N^{\zeta+1}\mathbb{Z})^\times \end{array} \right\}.$$

Proof. The following is an adaptation of the proof of security for Damgård–Jurik encryption [DJ01]. Define hybrid distributions $\mathcal{H}_i$ for $i \in \{0, \dots, \zeta\}$ as follows:

$$\mathcal{H}_i = \left\{ (N, x^{N^i}): \begin{array}{l} (N, p, q) \xleftarrow{\$} \text{RSA.Gen}(1^\lambda) \\ x \xleftarrow{\$} (\mathbb{Z}/N^{\zeta+1}\mathbb{Z})^\times \end{array} \right\}$$

Clearly, $\mathcal{H}_0$ and $\mathcal{H}_\zeta$ are our original left and right distributions, respectively.

To complete the proof, we must show that $\mathcal{H}_i \stackrel{c}{\approx} \mathcal{H}_{i+1}$ for all i . First, write $x = y \cdot \exp(Nr)$ for random $y \xleftarrow{\$} (\mathbb{Z}/N^{\zeta+1}\mathbb{Z})^\times$ and $r \xleftarrow{\$} \mathbb{Z}/N^\zeta\mathbb{Z}$. Notice that x only depends on y through $y \bmod N^2$, because $\exp(Nr)$ randomises the coefficients of N^2 and all higher powers in the N -adic expansion of x . (That is, $\exp(Nr)$ outputs a uniform element of $1 + N^2(\mathbb{Z}/N^{\zeta+1}\mathbb{Z})$.) Therefore, by DCR it is indistinguishable to set $y = y'^N$, where $y' \xleftarrow{\$} (\mathbb{Z}/N^{\zeta+1}\mathbb{Z})^\times$. Finally, let $x' = y' \exp(r)$. We now have

$$x^{N^i} = (y \exp(Nr))^{N^i} = (y'^N \exp(Nr))^{N^i} = (y' \exp(r))^{N^{i+1}} = x'^{N^{i+1}},$$

where the distribution of x' is uniform on $(\mathbb{Z}/N^{\zeta+1}\mathbb{Z})^\times$. □

A.2.3 KDM Security

Recall that we want to show that the following oracles

$\mathcal{O}_{\text{sk}, \text{pk}, R}^{\text{KDM}}(x, y):$ $(k, N) \leftarrow \text{sk}$ $z \leftarrow x \cdot k + y$ return $\text{DJE.Enc}_{\text{pk}}(z)$	$\mathcal{O}_{\text{sk}, \text{pk}, \$}^{\text{KDM}}(x, y):$ $(k, N) \leftarrow \text{sk}$ $z \xleftarrow{\$} \mathbb{Z}/N^\zeta \mathbb{Z}$ return $\text{DJE.Enc}_{\text{pk}}(z)$
--	--

are indistinguishable to a PPT adversary who is given pk but not sk . Here, $\text{sk} = (k, N)$ and $\text{pk} = (g, h, N)$ are sampled by generating a random $k \xleftarrow{\$} [0, N)$ and $g \xleftarrow{\$} (\mathbb{Z}/N^{\zeta+1}\mathbb{Z})^\times$, then setting $h \leftarrow g^{-k}$. We present a hybrid proof, starting with the real KDM oracle and ending at the random KDM oracle:

- $\mathcal{H}_1$. Sample g instead as $g = g'^{N^\zeta}$, where $g' \xleftarrow{\$} (\mathbb{Z}/N^{\zeta+1}\mathbb{Z})^\times$. This is indistinguishable by lemma 17.
- $\mathcal{H}_2$. Whenever Enc samples r from $[0, N)$, sample r from $[0, N^{\zeta+1}2^\lambda)$ instead. Note that r only matters modulo the order of g in $(\mathbb{Z}/N^{\zeta+1}\mathbb{Z})^\times$, and the order of g must divide $\varphi(N) = (p-1)(q-1)$ because g is a perfect N^ζ th power. Therefore, this change is statistically indistinguishable because both distributions for r are exponentially close to uniform modulo $\varphi(N)$:

$$\frac{N \bmod \varphi(N)}{N} = \frac{p+q-1}{N} = O(N^{-\frac{1}{2}}) \quad \text{and} \quad \frac{N^{\zeta+1}2^\lambda \bmod \varphi(N)}{N^{\zeta+1}2^\lambda} < N^{-\zeta}2^{-\lambda}.$$

- $\mathcal{H}_3$. Use lemma 17 to change g back to being sampled as $g \xleftarrow{\$} (\mathbb{Z}/N^{\zeta+1}\mathbb{Z})^\times$.
- $\mathcal{H}_4$. Sample $s \xleftarrow{\$} [0, N^\zeta)$ and add $\varphi(N)s$ to r inside Enc . That is, Enc will now calculate $c_0 \leftarrow g^{r+\varphi(N)s}$ and $c_1 \leftarrow h^{r+\varphi(N)s} \cdot \exp(z)$. This is statistically indistinguishable because r is sampled from a much wider range than this shift: $\frac{\varphi(N)N^\zeta}{N^{\zeta+1}2^\lambda} < 2^{-\lambda}$.
- $\mathcal{H}_5$. Rewrite $\mathcal{O}_{\text{sk}, \text{pk}, R}^{\text{KDM}}$ as follows, to avoid using sk :

$\mathcal{O}_{\text{sk}, \text{pk}, R}^{\text{KDM}}(x, y):$ $(g, h, N) \leftarrow \text{pk}$ $r \xleftarrow{\$} [0, N^{\zeta+1}2^\lambda)$ $s \xleftarrow{\$} [0, N^\zeta)$ $c_0 \leftarrow g^{r+\varphi(N)s} \cdot \exp(x)$ $c_1 \leftarrow h^{r+\varphi(N)s} \cdot \exp(y)$ return (c_0, c_1)

To see why this is indistinguishable, let $u = \log(g^{\varphi(N)})$ and notice that $-k \cdot u = \log(h^{\varphi(N)})$. Therefore, $c_0 = g^r \exp(u \cdot s)$ and $c_1 = h^r \exp(-k \cdot u \cdot s + x \cdot k + y)$. With probability $1 - N^{-1}$ we have $u \not\equiv 0 \pmod N$, so we can substitute $s = (s' + u^{-1}x) \bmod N^\zeta$ to get $c_0 = g^r \exp(u \cdot s' + x)$ and $c_1 = h^r \exp(-k \cdot u \cdot s' + y)$. Finally, changing back from $\exp(u)$ and $\exp(-k \cdot u)$ to $g^{\varphi(N)}$ and $h^{\varphi(N)}$, and renaming s' to s , gives the new KDM oracle above.

- $\mathcal{H}_6$. Undo the changes from $\mathcal{H}_4$, to remove s from the KDM oracle.
- $\mathcal{H}_7$. Once again, let $g = g'^{N^\zeta}$, where $g' \xleftarrow{\$} (\mathbb{Z}/N^{\zeta+1}\mathbb{Z})^\times$. This is indistinguishable by lemma 17.
- $\mathcal{H}_8$. Sample k from $[0, N^{\zeta+1}2^\lambda)$ instead $[0, N)$. This is statistically indistinguishable for the same reason as in $\mathcal{H}_2$, as k is now only used to compute $h = g^{-k}$.
- $\mathcal{H}_9$. Again, use lemma 17 to change g back to being sampled as $g \xleftarrow{\$} (\mathbb{Z}/N^{\zeta+1}\mathbb{Z})^\times$.
- $\mathcal{H}_{10}$. Add $\varphi(N)u^{-1} \bmod N^\zeta$ to k when computing h , where $u = \log(g^{\varphi(N)})$. That is, now $h = g^{-k+\varphi(N)u^{-1}} = g^{-k} \exp(1)$. This is statistically indistinguishable for the same reason as $\mathcal{H}_4$.
- $\mathcal{H}_{11}$. Again, use lemma 17 to set $g = g'^{N^\zeta}$, where $g' \xleftarrow{\$} (\mathbb{Z}/N^{\zeta+1}\mathbb{Z})^\times$.
- $\mathcal{H}_{12}$. Reintroduce s into the KDM oracle.²⁰ That is, sample $s \xleftarrow{\$} [0, N^\zeta)$ and add $\varphi(N)s$ to r . This is statistically

²⁰ We could not have kept s there the whole time, because the use of $\varphi(N)$ would interfere with using lemma 17 to change how g is sampled.

indistinguishable for the same reason as $\mathcal{H}_4$. Because the order of g divides $\varphi(N)$ (as g is a perfect N^ζ th power), we have

$$\begin{aligned} c_0 &= g^{r+\varphi(N)s} \cdot \exp(x) = g^r \exp(x) \\ c_1 &= h^{r+\varphi(N)s} \cdot \exp(y) = g^{-kr} \cdot \exp(\varphi(N)s + y). \end{aligned}$$

- $\mathcal{H}_{13}$. Replace y with $y' \xleftarrow{\$} \mathbb{Z}/N^\zeta\mathbb{Z}$. To show indistinguishability, substitute $s = (s' + \varphi(N)^{-1}(y' - y)) \bmod N^\zeta$, so that now $c_1 = g^{-kr} \cdot \exp(\varphi(N)s' + y')$. And sampling $s' \xleftarrow{\$} [0, N^\zeta)$ is an equivalent distribution to sampling s uniformly.
- $\mathcal{H}_{14}$. Now that y has been replaced with random, undo hybrids $\mathcal{H}_{12}$ – $\mathcal{H}_1$. We are now have a distribution equivalent to $\mathcal{O}_{\text{sk}, \text{pk}, R}^{\text{KDM}}$, except that it ignores y and replaces it with a uniformly sampled y' .
- $\mathcal{H}_{15}$. Notice that sampling $y' \xleftarrow{\$} \mathbb{Z}/N^\zeta\mathbb{Z}$ and computing $z \leftarrow x \cdot k + y'$ is equivalent to just sampling $z \xleftarrow{\$} \mathbb{Z}/N^\zeta\mathbb{Z}$, as the y' works as a one-time pad. Making this change, we are now at $\mathcal{O}_{\text{sk}, \text{pk}, \$}^{\text{KDM}}$.

A.3 Distributed Discrete Logarithm

DDLOG Damgård-Jurik Distance Function [RS21]

$\text{DDLog}_N(h \in \mathbb{Z}/N^{\zeta+1}\mathbb{Z})$:

Compute and output $z \leftarrow \log\left(\frac{h}{h \bmod N}\right) \in \mathbb{Z}/N^\zeta\mathbb{Z}$

Fig. 10: [RS21]’s distributed discrete logarithm for the Damgård-Jurik cryptosystem [DJ01].

Lemma 18 (Distributed Decryption). *If we have shares over $\mathbb{Z}$ of $\langle x \rangle_1 - \langle x \rangle_0 = x$ and $\langle k \cdot x \rangle_1 - \langle k \cdot x \rangle_0 = k \cdot x$, then*

$$\text{DDLog}_N(c_0^{\langle k \cdot x \rangle_1} c_1^{\langle x \rangle_1}) - \text{DDLog}_N(c_0^{\langle k \cdot x \rangle_0} c_1^{\langle x \rangle_0}) \equiv x \cdot y \pmod{N^\zeta}$$

always holds, for every choice of plaintext size $\zeta \geq 1$, key pair $(\text{sk} = (k, N), \text{pk}) \in \text{Supp}(\text{DJE.KeyGen}(1^\lambda))$, plaintext $y \in \mathbb{Z}/N^\zeta\mathbb{Z}$, ciphertext $(c_0, c_1) \in \text{Supp}(\text{DJE.Enc}_{\text{pk}}(y))$, and scalar $x \in \mathbb{Z}/N^\zeta\mathbb{Z}$.

Proof. Taking the ratio of the inputs to DDLog , we get

$$\frac{c_0^{\langle k \cdot x \rangle_1} c_1^{\langle x \rangle_1}}{c_0^{\langle k \cdot x \rangle_0} c_1^{\langle x \rangle_0}} = c_0^{k \cdot x} c_1^x = \exp(\text{DJE.Dec}_{\text{sk}}(c_0, c_1))^x = \exp(x \cdot y).$$

The second and third equalities are from the definition of DJE.Dec and the correctness of DJE , respectively. The inputs to the DDLogs are therefore multiplicative shares of $\exp(x \cdot y)$, which, by [RS21, Theorem 18], makes the outputs of the DDLogs additive shares over $\mathbb{Z}/N^\zeta\mathbb{Z}$ of $x \cdot y$. $\square$

Lemma 19 (Adapted from [RS21, Lemma 19]). *For all moduli $M > 1$ and all modulo M shares $\langle x \rangle_0, \langle x \rangle_1 \in \mathbb{Z}/M\mathbb{Z}$ of some $x \in \mathbb{Z}$, we have*

$$\Pr_{r \xleftarrow{\$} \mathbb{Z}/M\mathbb{Z}} \left[(\langle x \rangle_1 + r) \bmod M - (\langle x \rangle_0 + r) \bmod M = x \right] = \max \left(1 - \frac{|x|}{N^\zeta}, 0 \right).$$

B Proofs

B.1 Proof of core lemma 1

The proof of core lemma 1 boils down to proving that all the comments in the pseudocode of `ShareConv` are invariants holding with all but negligible probability.

These claims are:

$$\text{For } i \in [r]: \text{ct}'_i \in \text{Supp}(\text{Enc}(k \cdot f'_i(\text{sh}_{0,1}, \dots, \text{sh}_{0,m}))) \quad (5)$$

$$\text{For } i \in [r]: \text{ct}''_i \in \text{Supp}(\text{Enc}(f'_i(\text{sh}_{0,1}^{\text{auth}}, \dots, \text{sh}_{0,m}^{\text{auth}}))) \quad (6)$$

$$\text{For } i \in [r]: \text{ct}'''_i \in \text{Supp}(\text{Enc}(f_i(-\text{sh}_{0,1}, \dots, -\text{sh}_{0,m}))) \quad (7)$$

$$\text{For } i \in [r]: \text{ct}''''_i \in \text{Supp}(\text{Enc}(k \cdot f_i(-\text{sh}_{0,1}, \dots, -\text{sh}_{0,m}))) \quad (8)$$

For $i \in [r]$, $j \in [0, d_i]$, and $\ell \in [d_i - j]$:

$$\text{TAB}_1[i][j][\ell] - \text{TAB}_0[i][j][\ell] = [f'_i(\text{sh}_{1,1}, \dots, \text{sh}_{1,m})]^j \cdot [f_i(-\text{sh}_{0,1}, \dots, -\text{sh}_{0,m})]^\ell \quad (9)$$

For $i \in [r]$, $j \in [0, d_i]$, and $\ell \in [d_i - j]$:

$$\text{TAB}_1^{\text{auth}}[i][j][\ell] - \text{TAB}_0^{\text{auth}}[i][j][\ell] = k \cdot [f'_i(\text{sh}_{1,1}, \dots, \text{sh}_{1,m})]^j \cdot [f_i(-\text{sh}_{0,1}, \dots, -\text{sh}_{0,m})]^\ell \quad (10)$$

$$y_1 - y_0 = f(x_1, \dots, x_n) \quad (11)$$

$$y_1^{\text{auth}} - y_0^{\text{auth}} = k \cdot f(x_1, \dots, x_n) \quad (12)$$

Note that eqs. (11) and (12) imply our desired equalities, because if $y_1 - y_0 \in \mathbb{Z}$ then $\lfloor y_1 \rfloor - \lfloor y_0 \rfloor = y_1 - y_0$. Let us now prove that these invariants all hold with the desired probability. We do this by first establishing a sequence of problem-reducing implications between these various statements. First observe that for all $i \in [r]$, $|f(\text{sh}_{0,1}, \dots, \text{sh}_{0,m})|, |f'(\text{sh}_{0,1}, \dots, \text{sh}_{0,m})|, |f(\text{sh}_{1,1}, \dots, \text{sh}_{1,m})|, |f'(\text{sh}_{1,1}, \dots, \text{sh}_{1,m})| \leq m \cdot B \cdot 2^\lambda \cdot \beta$. In particular, because $k \leq N$, $k \cdot |f(\text{sh}_{0,1}, \dots, \text{sh}_{0,m})|, k \cdot |f'(\text{sh}_{0,1}, \dots, \text{sh}_{0,m})|, k \cdot |f(\text{sh}_{1,1}, \dots, \text{sh}_{1,m})|, k \cdot |f'(\text{sh}_{1,1}, \dots, \text{sh}_{1,m})| \leq N \cdot m \cdot B \cdot 2^\lambda \cdot \beta$.

1. **Equations (5) to (8) hold unconditionally for all $i \in [r]$.** For each $i \in [r]$, eqs. (5) and (6) holding (with probability 1) follow directly from linear homomorphism of the Damgård-Jurik-ElGamal encryption scheme. Equations (7) and (8) also follow from linear homomorphism, with the observation that $g_i(\text{sh}_{0,1}, \dots, \text{sh}_{0,m}, 1) = f'_i(\text{sh}_{0,1}, \dots, \text{sh}_{0,m}) + f_i(0, \dots, 0) = f_i(\text{sh}_{0,1}, \dots, \text{sh}_{0,m})$ (for eq. (7)) and $g_i(k \cdot \text{sh}_{0,1}, \dots, k \cdot \text{sh}_{0,m}, k) = f'_i(k \cdot \text{sh}_{0,1}, \dots, k \cdot \text{sh}_{0,m}) + f_i(0, \dots, 0) \cdot k = k \cdot (f_i(\text{sh}_{0,1}, \dots, \text{sh}_{0,m}) - f(0, \dots, 0)) + f(0, \dots, 0) \cdot k = k \cdot f(\text{sh}_{0,1}, \dots, \text{sh}_{0,m})$ (for eq. (8)).
2. **For all $i \in [r]$, if $j = 0$ and $\ell = d_i$, eqs. (9) and (10) holds unconditionally.** This is true tautologically for all $i \in [r]$ because $\text{TAB}_1[i][0][d_i]$ is defined as 0 and $\text{TAB}_0[i][0][d_i]$ is defined as $[f_i(-\text{sh}_{0,1}, \dots, -\text{sh}_{0,m})]^{d_i}$, and because $\text{TAB}_1^{\text{auth}}[i][0][d_i]$ is defined as 0 and $\text{TAB}_0^{\text{auth}}[i][0][d_i]$ is defined as $k \cdot [f_i(-\text{sh}_{0,1}, \dots, -\text{sh}_{0,m})]^{d_i}$.
3. **If $\ell = 0$, eq. (9) holds unconditionally for all $i \in [r]$ and $j \in [d_i]$.** If $\ell = 0$, then eq. (9) is true tautologically for all $i \in [r]$ and $j \in [d_i]$ because $\text{TAB}_1[i][j][0]$ is defined as $[f'_i(\text{sh}_{1,1}, \dots, \text{sh}_{1,m})]^j$ and $\text{TAB}_0[i][j][0]$ is defined as 0.

4. If $\ell = 0$ and $j = 1$, eq. (10) holds unconditionally for all $i \in [r]$. For all $i \in [r]$,

$$\begin{aligned}
\text{TAB}_1^{\text{auth}}[i][1][0] - \text{TAB}_0^{\text{auth}}[i][1][0] &= f'_i(\text{sh}_{1,1}^{\text{auth}}, \dots, \text{sh}_{1,m}^{\text{auth}}) - f'_i(\text{sh}_{0,1}^{\text{auth}}, \dots, \text{sh}_{0,m}^{\text{auth}}) \\
&\quad + k \cdot f'_i(\text{sh}_{0,1}, \dots, \text{sh}_{0,m}) \\
&= f'_i(\text{sh}_{1,1}^{\text{auth}} - \text{sh}_{0,1}^{\text{auth}}, \dots, \text{sh}_{1,m}^{\text{auth}} - \text{sh}_{0,m}^{\text{auth}}) \\
&\quad + k \cdot f'_i(\text{sh}_{0,1}, \dots, \text{sh}_{0,m}) \\
&= f'_i(k \cdot x_1, \dots, k \cdot x_n) + k \cdot f'_i(\text{sh}_{0,1}, \dots, \text{sh}_{0,m}) \\
&= k \cdot f'_i(x_1, \dots, x_n) + k \cdot f'_i(\text{sh}_{0,1}, \dots, \text{sh}_{0,m}) \\
&= k \cdot f'_i(\text{sh}_{1,1} - \text{sh}_{0,1}, \dots, \text{sh}_{1,m} - \text{sh}_{0,m}) \\
&\quad + k \cdot f'_i(\text{sh}_{0,1}, \dots, \text{sh}_{0,m}) \\
&= k \cdot f'_i(\text{sh}_{1,1}, \dots, \text{sh}_{1,m})
\end{aligned}$$

by linearity of f'_i and the facts that $\text{sh}_{1,t} - \text{sh}_{0,t} = x_t$ and $\text{sh}_{1,t}^{\text{auth}} - \text{sh}_{0,t}^{\text{auth}} = k \cdot x_t$. Hence eq. (10) holds unconditionally for all $i \in [r]$ is $(j, \ell) = (1, 0)$.

5. If $\ell = 0$, then for all $i \in [r]$ and $j \in [2, d_i]$, if eq. (10) holds for $(i, j-1, \ell = 0)$ then it holds with probability at least $1 - 4(nB2^\lambda\beta)^d/N^{\zeta-1}$ for $(i, j, \ell = 0)$. For all $i \in [r]$ and $j \in [2, d_i]$, if we assume eq. (10) hold for $(i, j-1, \ell = 0)$ then, because , the following hold with probability at least $1 - 4(nB2^\lambda\beta)^d/N^{\zeta-1}$ by lemma 6 (using eqs. (5) and (6)):

$$\begin{aligned}
\text{TAB}_1^{\text{auth}}[i][j][0] - \text{TAB}_0^{\text{auth}}[i][j][0] &= f'_i(\text{sh}_{1,1}^{\text{auth}}, \dots, \text{sh}_{1,m}^{\text{auth}}) \cdot [f'_i(\text{sh}_{1,1}, \dots, \text{sh}_{1,m})]^{j-1} \\
&\quad + \text{Mult}(\text{TAB}_1[i][j-1][0], \text{TAB}_1^{\text{auth}}[i][j-1][0], \text{ct}'_i, \text{crs}_{i,j,1}) \\
&\quad - \text{Mult}(\text{TAB}_1[i][j-1][0], \text{TAB}_1^{\text{auth}}[i][j-1][0], \text{ct}''_i, \text{crs}_{i,j,2}) \\
&\quad - \text{Mult}(\text{TAB}_0[i][j-1][0], \text{TAB}_0^{\text{auth}}[i][j-1][0], \text{ct}'_i, \text{crs}_{i,j,1}) \\
&\quad + \text{Mult}(\text{TAB}_0[i][j-1][0], \text{TAB}_0^{\text{auth}}[i][j-1][0], \text{ct}''_i, \text{crs}_{i,j,2}) \\
&= f'_i(\text{sh}_{1,1}^{\text{auth}}, \dots, \text{sh}_{1,m}^{\text{auth}}) \cdot [f'_i(\text{sh}_{1,1}, \dots, \text{sh}_{1,m})]^{j-1} \\
&\quad + [f'_i(\text{sh}_{1,1}, \dots, \text{sh}_{1,m})]^{j-1} \cdot k \cdot f'_i(\text{sh}_{0,1}, \dots, \text{sh}_{0,m}) \\
&\quad - [f'_i(\text{sh}_{1,1}, \dots, \text{sh}_{1,m})]^{j-1} \cdot f'_i(\text{sh}_{0,1}^{\text{auth}}, \dots, \text{sh}_{0,m}^{\text{auth}}) \\
&= [f'_i(\text{sh}_{1,1}, \dots, \text{sh}_{1,m})]^{j-1} \\
&\quad \cdot [f'_i(\text{sh}_{1,1}^{\text{auth}}, \dots, \text{sh}_{1,m}^{\text{auth}}) + k \cdot f'_i(\text{sh}_{0,1}, \dots, \text{sh}_{0,m}) - f'_i(\text{sh}_{0,1}^{\text{auth}}, \dots, \text{sh}_{0,m}^{\text{auth}})] \\
&= [f'_i(\text{sh}_{1,1}, \dots, \text{sh}_{1,m})]^{j-1} \\
&\quad \cdot [f'_i(k \cdot x_1, \dots, k \cdot x_n) + k \cdot f'_i(\text{sh}_{0,1}, \dots, \text{sh}_{0,m})] \\
&= [f'_i(\text{sh}_{1,1}, \dots, \text{sh}_{1,m})]^{j-1} \\
&\quad \cdot k \cdot f'_i(x_1 + \text{sh}_{0,1}, \dots, x_n + \text{sh}_{0,m}) \\
&= k \cdot [f'_i(\text{sh}_{1,1}, \dots, \text{sh}_{1,m})]^j
\end{aligned}$$

6. For all $i \in [r]$, $j \in [d_i]$, and $\ell \in [d_i - j]$, if eqs. (9) and (10) hold for $(i, j, \ell - 1)$ then they hold with probability at least $1 - 2(nB2^\lambda\beta)^d/N^{\zeta-1}$ for $(i, j, \ell - 1)$. Let $(i, j, \ell) \in [r] \times [d_i] \times [d_i - j]$ and assume eqs. (9) and (10) both hold.

By lemma 6 (and using eq. (7)), with all but probability at most $2(nB2^\lambda\beta)^d/N^{\zeta-1}$:

$$\begin{aligned} \text{TAB}_1[i][j][\ell] - \text{TAB}_0[i][j][\ell] &= \text{Mult}(\text{TAB}_1[i][j][\ell-1], \text{TAB}_1^{\text{auth}}[i][j][\ell-1], \text{ct}_i''', \text{crs}_{i,j,3}) \\ &\quad - \text{Mult}(\text{TAB}_1[i][j][\ell-1], \text{TAB}_1^{\text{auth}}[i][j][\ell-1], \text{ct}_i''', \text{crs}_{i,j,3}) \\ &= [f'_i(\text{sh}_{1,1}, \dots, \text{sh}_{1,m})]^j \cdot [f_i(-\text{sh}_{0,1}, \dots, -\text{sh}_{0,m})]^{\ell-1} \cdot f_i(-\text{sh}_{0,1}, \dots, -\text{sh}_{0,m}) \\ &= [f'_i(\text{sh}_{1,1}, \dots, \text{sh}_{1,m})]^j \cdot [f_i(-\text{sh}_{0,1}, \dots, -\text{sh}_{0,m})]^\ell \end{aligned}$$

which is to say eq. (9) holds for (i, j, ℓ) .

Similarly, by lemma 6 (and using eq. (8)), with probability at least $1 - 2B/N^\varepsilon$:

$$\begin{aligned} \text{TAB}_1^{\text{auth}}[i][j][\ell] - \text{TAB}_0^{\text{auth}}[i][j][\ell] &= \text{Mult}(\text{TAB}_1[i][j][\ell-1], \text{TAB}_1^{\text{auth}}[i][j][\ell-1], \text{ct}_i''', \text{crs}_{i,j,4}) \\ &\quad - \text{Mult}(\text{TAB}_1[i][j][\ell-1], \text{TAB}_1^{\text{auth}}[i][j][\ell-1], \text{ct}_i''', \text{crs}_{i,j,4}) \\ &= [f'_i(\text{sh}_{1,1}, \dots, \text{sh}_{1,m})]^j \cdot [f_i(-\text{sh}_{0,1}, \dots, -\text{sh}_{0,m})]^{\ell-1} \cdot k \cdot f_i(-\text{sh}_{0,1}, \dots, -\text{sh}_{0,m}) \\ &= k \cdot [f'_i(\text{sh}_{1,1}, \dots, \text{sh}_{1,m})]^j \cdot [f_i(-\text{sh}_{0,1}, \dots, -\text{sh}_{0,m})]^\ell \end{aligned}$$

which is to say eq. (10) holds for (i, j, ℓ) .

7. *If, for all $i \in [r]$, $j \in [d_i]$, eq. (9) holds for $(i, j, \ell = d_i - j)$, then eq. (11) holds.* Assume eq. (9) holds for $(i, j, \ell = d_i - j)$ for all $(i, j) \in [r] \times [d_i]$, then

$$\begin{aligned} y_1 - y_0 &= \sum_{i=1}^r \sum_{j=1}^{d_i} \alpha_i \cdot \binom{d_i}{j} \cdot (-1)^{d_i-j} \cdot (\text{TAB}_1[i][j][d_i-j] - \text{TAB}_0[i][j][d_i-j]) \\ &= \sum_{i=1}^r \sum_{j=1}^{d_i} \alpha_i \cdot \binom{d_i}{j} \cdot (-1)^{d_i-j} \cdot [f'_i(\text{sh}_{1,1}, \dots, \text{sh}_{1,m})]^j \cdot [f_i(-\text{sh}_{0,1}, \dots, -\text{sh}_{0,m})]^{d_i-j} \\ &= \sum_{i=1}^r \alpha_i \cdot (f'_i(\text{sh}_{1,1}, \dots, \text{sh}_{1,m}) - f_i(-\text{sh}_{0,1}, \dots, -\text{sh}_{0,m}))^{d_i} \\ &= \sum_{i=1}^r \alpha_i \cdot (f_i(x_1, \dots, x_n))^{d_i} \\ &= f(x) \end{aligned}$$

That is to say, eq. (11) holds.

8. *If, for all $i \in [r]$, $j \in [d_i]$, eq. (10) holds for $(i, j, \ell = d_i - j)$, then eq. (12) holds.* Assume eq. (10) holds for $(i, j, \ell = d_i - j)$ for all $(i, j) \in [r] \times [d_i]$, then

$$\begin{aligned} y_1^{\text{auth}} - y_0^{\text{auth}} &= \sum_{i=1}^r \sum_{j=0}^{d_i} \alpha_i \cdot \binom{d_i}{j} \cdot (-1)^{d_i-j} \cdot (\text{TAB}_1^{\text{auth}}[i][j][d_i-j] - \text{TAB}_0^{\text{auth}}[i][j][d_i-j]) \\ &= \sum_{i=1}^r \sum_{j=0}^{d_i} \alpha_i \cdot \binom{d_i}{j} \cdot (-1)^{d_i-j} \cdot k \cdot [f'_i(\text{sh}_{1,1}, \dots, \text{sh}_{1,m})]^j \cdot [f_i(-\text{sh}_{0,1}, \dots, -\text{sh}_{0,m})]^{d_i-j} \\ &= k \cdot \sum_{i=1}^r \alpha_i \cdot (f'_i(\text{sh}_{1,1}, \dots, \text{sh}_{1,m}) - f_i(-\text{sh}_{0,1}, \dots, -\text{sh}_{0,m}))^{d_i} \\ &= k \cdot \sum_{i=1}^r \alpha_i \cdot (f_i(x_1, \dots, x_n))^{d_i} \\ &= k \cdot f(x) \end{aligned}$$

That is to say, eq. (12) holds.

By combining these polynomially many implications, we get that eqs. (11) and (12) hold simultaneously with probability at least $1 - 4rd^2(nB2^\lambda\beta)^d/N^{\zeta-1}$ (by a union bound), which is what we set out to prove.

B.2 Proof of main theorem 1

In this section we prove main theorem 1. Even though definition 5 is simulation-based, we start by establishing a strong notion of correctness in section B.2.1 for convenience, before resuming the proof in section B.2.2.

B.2.1 Correctness lemma.

Lemma 20 (Correctness of the PCPRF of fig. 8). *Let $\lambda \in \mathbb{N}^*$ be a security parameter, let $(\lambda_{\text{DCE}}, \zeta)$ be parameters for the Damgård-Jurik-ElGamal cryptosystem, let B be a bound on inputs. Let r, d, β be polynomial-size bounds. Consider the construction of fig. 8 for any class of constraints C whose dual have Waring rank bounded by r and degree bounded by d , and furthermore such that the coefficients of affine functions used in a corresponding Waring decomposition (i.e. “the f_i ”) are bounded by β .*

$\forall C \in \mathcal{C}$,

$$\Pr \left[\begin{array}{l} \forall x \in \{0, 1\}^m, \forall k_{\text{PRF}} \in \{0, 1\}^\lambda, \text{ } B\text{-admissible w.r.t. PRF, } \\ \text{CEval}(1^\lambda, \text{ck}, x) - \text{Eval}(1^\lambda, \text{msk}, x) = C(x) \cdot \text{PRF}_{k_{\text{PRF}}}(x) \end{array} : \begin{array}{l} \text{msk} \xleftarrow{\$} \text{PCPRF.KeyGen}(1^\lambda) \\ \text{ck} \xleftarrow{\$} \text{PCPRF.Constrain}(1^\lambda, \text{msk}, C) \end{array} \right] \\ \geq 1 - \frac{2|\text{PRF}| \cdot B + 4rd^2(nB2^\lambda\beta)^d}{(2^{\lambda_{\text{DCE}}-1})^{\zeta-1}} \cdot 2^{n\lambda}.$$

Proof. Let $C \in \mathcal{C}$, $x \in \{0, 1\}^m$, and $k_{\text{PRF}} \in \{0, 1\}^\lambda$ which is B -admissible with respect to bound B . By combining core lemma 1 and lemma 8, we get that

$$\Pr \left[\text{CEval}(1^\lambda, \text{ck}, x) - \text{Eval}(1^\lambda, \text{msk}, x) = C(x) \cdot \text{PRF}_{k_{\text{PRF}}}(x) : \begin{array}{l} \text{msk} \xleftarrow{\$} \text{PCPRF.KeyGen}(1^\lambda) \\ \text{ck} \xleftarrow{\$} \text{PCPRF.Constrain}(1^\lambda, \text{msk}, C) \end{array} \right] \\ \geq 1 - \frac{2|\text{PRF}| \cdot B + 4rd^2(nB2^\lambda\beta)^d}{(2^{\lambda_{\text{DCE}}-1})^{\zeta-1}}.$$

We get the desired result by a union bound over all x and k_{PRF} . $\square$

B.2.2 Full simulation-based proof. We now prove single-key selective PCPRF security, using the strong correctness (lemma 20) of our PCPRF, and the KDM security of the underlying Damgård–Jurik–ElGamal encryption scheme (theorem 16). We must show that the constrained key can be simulated using only the size of the constraint, such that oracle access to master evaluations $\text{Eval}(1^\lambda, \text{msk}, \cdot)$ in the real world is indistinguishable from oracle access to

$$x \mapsto \begin{cases} \text{CEval}(1^\lambda, \text{ck}, x) & \text{if } C(x) = 0 \\ R(x) & \text{if } C(x) = 1 \end{cases},$$

in the simulated world, where $R: \{0, 1\}^n \rightarrow [0, 2^\mu)$ is sampled as a uniformly random function. In fig. 11 we present our simulator for this property.

Simulator Privately Constrained PRF

$\mathcal{S}(1^\lambda, 1^m)$:

1. $(\text{sk} = (k, N), \text{pk} = (g, h, N)) \xleftarrow{\$} \text{DJE.KeyGen}(1^\lambda)$
2. For $i \in [m]$:
 - $\text{sh}_{1,i} \xleftarrow{\$} [2^\lambda]$

- $\text{sh}_{1,i}^{\text{auth}} \xleftarrow{\$} [N2^\lambda]$
- $c_i, c_i^{\text{auth}}, c_i^{\text{ext-auth}} \xleftarrow{\$} \text{DJE.Enc}_{\text{pk}}(\$)$ // All are encryptions of uniformly random plaintexts.

3. For $i \in [\lambda]$:

- $c_{\text{PRF},i} \xleftarrow{\$} \text{DJE.Enc}_{\text{pk}}(\$)$
- $c_{\text{PRF},i}^{\text{ext-auth}} \xleftarrow{\$} \text{DJE.Enc}_{\text{pk}}(\$)$

4. $\text{crs} = (\text{crs}_1, \text{crs}_2) \xleftarrow{\$} (\mathbb{Z}/N^\zeta\mathbb{Z})^{4 \cdot rd} \times (\mathbb{Z}/N^\zeta\mathbb{Z})^M$

5. $\text{ck} \leftarrow (\text{pk},$
 $(\text{sh}_{1,i}, \text{sh}_{1,i}^{\text{auth}}, c_i, c_i^{\text{auth}}, c_i^{\text{ext-auth}})_{i \in [m]},$
 $(c_{\text{PRF},i}, c_{\text{PRF},i}^{\text{ext-auth}})_{i \in [\lambda]},$
 $\text{crs})$

6. Output ck

Fig. 11: Simulator for single-key selective security.

We now present a hybrid proof starting from the real world and ending at the simulated world.

$\mathcal{H}_1$. First, we use lemma 20 to rewrite the oracle access to $\text{Eval}(1^\lambda, \text{msk}, \cdot)$ in terms of $\text{PEval}(1^\lambda, \text{ck}, \cdot)$. According to the lemma, λ_{DJE} and ζ can be chosen as sufficiently large polynomials in the security parameter so that except with negligible probability we have that, for all evaluation points x ,

$$\text{CEval}(1^\lambda, \text{ck}, x) - \text{Eval}(1^\lambda, \text{msk}, x) \equiv C(x) \cdot \text{PRF}_{k_{\text{PRF}}}(x) \pmod{2^\mu}.$$

Therefore, we rewrite the Eval oracle as

$$x \mapsto (\text{CEval}(1^\lambda, \text{ck}, x) - C(x) \cdot \text{PRF}_{k_{\text{PRF}}}(x)) \pmod{2^\mu}.$$

$\mathcal{H}_2$. For all $i \in [m]$, instead of sampling $\text{sh}_{0,i} \xleftarrow{\$} [2^\lambda]$ and setting $\text{sh}_{1,i} \leftarrow \widehat{C}_i + \text{sh}_{0,i}$, sample $\text{sh}_{1,i} \xleftarrow{\$} [2^\lambda]$ and set $\text{sh}_{0,i} \leftarrow -\widehat{C}_i + \text{sh}_{1,i}$. This shifts the distribution of the shares by at most 1, so each change gives an advantage of at most $2^{-\lambda}$. Similarly, we reverse which share is sampled first for the auth shares: now $\text{sh}_{1,i}^{\text{auth}} \xleftarrow{\$} [N2^\lambda]$ and $\text{sh}_{0,i}^{\text{auth}} \leftarrow -k \cdot \widehat{C}_i + \text{sh}_{1,i}^{\text{auth}}$. This shifts the shares by at most $k \leq N$, so again each change has an advantage of at most $2^{-\lambda}$. The total advantage from this hybrid is then at most $m2^{1-\lambda}$.

$\mathcal{H}_3$. Notice that there are now only two places where k is used after DJE.KeyGen : (i) to compute master key auth shares $\text{sh}_{0,\dots}^{\text{auth}}$, which only used to create ciphertexts $c_{\dots}^{\text{auth}} \xleftarrow{\$} \text{DJE.Enc}_{\text{pk}}(\text{sh}_{0,\dots}^{\text{auth}})$, and (ii) to generate the ext-auth ciphertexts $c_{\dots}^{\text{ext-auth}} \xleftarrow{\$} \text{DJE.Enc}_{\text{pk}}(k \cdot \text{sh}_{0,\dots})$. Both of these are only encryptions of affine functions of the key, so by the KDM security of DJE it is indistinguishable to replace all ciphertexts with encryptions of uniformly random values.

$\mathcal{H}_4$. Notice that the master key shares $\text{sh}_{0,\dots}$ and $\text{sh}_{0,\dots}^{\text{auth}}$ are no longer used. Remove them.

$\mathcal{H}_5$. Notice that the PRF key k_{PRF} is now only used in the Eval oracle. Therefore, by security of the PRF we can remove k_{PRF} , and instead sample a uniformly random function $R': \{0, 1\}^n \rightarrow [0, 2^\mu]$ and let the Eval oracle be

$$x \mapsto (\text{CEval}(1^\lambda, \text{ck}, x) - C(x) \cdot R'(x)) \pmod{2^\mu}.$$

$\mathcal{H}_6$. Define $R(x) = (\text{CEval}(1^\lambda, \text{ck}, x) - R'(x)) \pmod{2^\mu}$. Then we can write the Eval oracle as

$$x \mapsto \begin{cases} \text{CEval}(1^\lambda, \text{ck}, x) & \text{if } C(x) = 0 \\ R(x) & \text{if } C(x) = 1 \end{cases}.$$

$\mathcal{H}_7$. Because R' has outputs uniform on $[0, 2^\mu]$, it is an identical distribution to remove R' and directly sample R uniformly. This hybrid matches the simulation.